DMDG ENTERPRISE MASTER MANUAL #04 — 완전가이드

[스피디] 공식 실전 구축 엔지니어링 바이블

클라우드 EDR 악성코드 조기 예측 및 가상 샌드박스 동적 행위 탐지

0.05초 정적 PE 엔트로피 분석 & LightGBM 98.5% 예측 및 Gemini MITRE ATT&CK/YARA 자동화

일반인도 책만 펼쳐놓고 그대로 따라 치며 시스템을 직접 구축하는 대한민국 최고봉 실무 전공서

알려지지 않은 신종 랜섬웨어와 지능형 사이버 위협을 0.05초 만에 사전 예측하고 도커 가상 샌드박스에서 격리 분석하는 차세대 보안 관제 센터(SOC) 실습서입니다. 33개 챕터 전수 수록.

- 실시간 센서 인제스천 ➔ 수치 연산 코어 ➔ Google Gemini LLM 오케스트레이션 ➔ 물리 액추에이터 제어
- 라인 바이 라인 전수 소스코드 분석, 수학적 공식 유도, 프로덕션 트러블슈팅 런북, 부하 테스트 완전 수록
- 순수 출판 무결점 에디션 (상업용 웹 후원 배너 100% 완전 제거 및 인쇄 가독성 극대화)

기획 및 집필: 당목담글(DMDG) 스튜디오 Great AI 출판본부

저자: Hamilton.Kang & DMDG 어벤져스 군단

발행처: DMDG Publishing & dmdg-live.com

AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템 구축

부제: [기업맞춤형 - 스피디] 대규모 데이터셋(EMBER)과 샌드박싱 기술을 활용한 지능형 악성코드 조기 차단 및 EDR 관제 솔루션

저자: DMDG AI Tech Lab (feat. 주식회사 스피디 사이버 보안 엔지니어링 도메인)

출판: 2026 Edition (위키독스 공식 오픈 교재)

💡 이 도서의 핵심 라이브러리 & 사이버 보안 엔지니어링 용어 사전

🛠️ 핵심 라이브러리 & 기술 스택

- `pefile / capstone` : Windows PE 바이너리 정적 구조 파싱 및 Opcode n-gram 역어셈블 엔진
- `scikit-learn / XGBoost / LightGBM` : 정적 피처 기반 고속 악성코드 분류 앙상블 머신러닝 모델
- `PyTorch (MalConv 1D-CNN & Bi-LSTM)` : 원시 바이트(Raw Byte)를 직접 임베딩하여 읽는 심층 신경망 모델링
- `Docker & API Hooking` : 안전한 가상 격리 샌드박스 런타임 및 실시간 시스템 콜 가로채기
- `Watchdog & Streamlit 1.40+` : 엔드포인트 파일시스템 실시간 감시 데몬 및 다크 사이버 SOC 관제 UI
- `YARA` : 신종 악성 패턴 자동 추출 및 시그니처 룰 생성 엔진

📖 필수 사이버 보안 핵심 용어

- **PE (Portable Executable)** : 윈도우 실행 파일(`.exe`, `.dll`)의 표준 바이너리 파일 포맷
- **엔트로피 (Byte Entropy)** : 바이트 분포의 무작위성 척도로 패킹(UPX) 및 랜섬웨어 암호화 여부를 즉시 판정하는 척도
- **EDR (Endpoint Detection & Response)** : 단말기(PC/서버)의 실시간 행위를 감시하고 위협 포착 시 즉시 격리하는 엔드포인트 보안 시스템
- **샌드박싱 (Sandboxing)** : 호스트 OS에 영향을 주지 않는 완전 격리된 가상 환경에서 악성코드를 안전하게 실행 및 행위 추적하는 기법
- **FPR (False Positive Rate, 오탐률)** : 정상 소프트웨어를 악성으로 오인하여 차단하는 비율 (산업 보안 솔루션 기준 0.1% 미만 필수)

- **C2 (Command and Control)** : 감염된 PC가 해커의 외부 원격 명령을 수신하기 위해 통신하는 악성 제어 서버

📖 도서 및 프로젝트 소개

최근 사이버 공격의 90% 이상은 기존 백신이 보유한 해시값(MD5, SHA-256) 데이터베이스를 완전히 우회하는 **신종·변종 악성코드(Zero-day Malware)**입니다. 해커들은 자동화된 패커(UPX, Themida), 다형성(Polymorphism), 파일리스(Fileless) 메모리 인젝션 기법을 사용하여 1초마다 파일 해시를 변경합니다.

본 도서는 차세대 보안 솔루션 전문 기업 **(주)스피디(Speedy)**의 엔지니어링 수요에 맞춰 기획된 **기업 맞춤형 AI-EDR 개발 실무 프로젝트**입니다.

파일 실행 전 0.05초 만에 위험을 감지하는 **1단계 초고속 정적 스크리닝(PE 헤더 & MalConv)**부터, 의심 파일을 격리된 컨테이너에서 가상 실행하여 랜섬웨어 암호화와 C2 통신을 적발하는 **2단계 동적 샌드박싱**, 그리고 실시간 파일시스템 감시와 원클릭 격리를 수행하는 **Streamlit SOC 보안 관제 센터**까지 풀사이클(End-to-End)로 구현합니다.

🎯 8대 핵심 엔지니어링 프로세스

본 과정은 기업(스피디)의 8대 핵심 요구사항을 100% 만족하도록 설계되었습니다:

1. **대규모 데이터셋 수집**: 글로벌 표준 EMBER(110만 건) / SOREL-20M 오픈셋 + 정상 바이너리 50,000건 확보
2. **악성/정상 구별 특징 추출**: PE Header, 섹션 엔트로피, 위험 Import API(`VirtualAllocEx`), Opcode n-gram
3. **머신러닝 & 딥러닝 알고리즘 적용**: SVM, Random Forest, XGBoost, Raw Byte 기반 1D-CNN MalConv 모델링
4. **알고리즘 모델 학습 및 평가**: 탐지 정확도 F1-Score 0.98, 오탐률(FPR) 0.1% 이하 억제 최적화
5. **샌드박싱 기술 활용 동적 분석**: Docker 가상 격리 환경에서 가상 실행 후 API Call 시퀀스 및 네트워크 C2 패킷 추적
6. **정상 행위 모델링**: 화이트리스트 정상 프로세스의 시스템 콜 베이스라인 프로파일링 (One-Class SVM)
7. **실시간 감지 시스템 구현**: `Watchdog` 기반 실시간 파일 감시 데몬 ➔ 의심 파일 0.05초 내 1차 판정 및 격리 큐 이관
8. **패턴 지속 업데이트 MLOps**: 신종 악성코드 발견 시 `YARA` 룰 자동 생성 및 주간 단위 지속적 재학습 파이프라인

📂 대규모 데이터셋 수집 및 구축 전략

실제 수만 개의 악성 바이너리를 로컬 PC에 직접 다운로드하는 것은 보안 정책(Windows Defender 삭제) 및 감염 위험이 발생하므로, 본 프로젝트는 **글로벌 보안 산업계 표준인 2-Track 전략**을 적용합니다:

```
[데이터셋 구축 2-Track 전략]

Track 1: 대규모 AI 모델 학습용 (50,000 ~ 100,000건)
 └── EMBER (Elastic) & SOREL-20M (Sophos)의 "사전 벡터화 피처셋(Vectorized Features)" 활용
```

```
  └── 장점: 감염 위험 0%, 수 GB 수준으로 가벼움, 학생 PC 사양 무관 초고속 학습 가능
  └── 정상 파일: EMBER 내장 40만 건 클린 데이터 + Windows System32 정품 바이너리

Track 2: Docker 샌드박스 동적 행위 분석용 (실습용 50 ~ 100건)
  └── MalwareBazaar (abuse.ch) 무료 REST API를 통해 암호화(infected)된 실제 샘플 50건 선별 수집
  └── 호스트 OS가 아닌 "도커 격리 컨테이너 내부"로만 마운트하여 안전 실행 및 API 캡처
```

🏗️ 시스템 아키텍처 및 2단계 하이브리드 파이프라인

```
[엔드포인트 실시간 감시 (Watchdog Daemon)]
   | (신규 파일 다운로드 / 프로세스 생성 실시간 감지)
   ▼
[1단계: 초고속 정적 예측 (Static Screening)]  ──< 0.05초 소요 >──
   ├─► PE 헤더 / 섹션 엔트로피 / 위험 API 특징 추출 (pefile)
   ├─► Random Forest + MalConv 딥러닝 앙상블 판정
   |
   ├─► [확정 정상 (Clean, 확률 > 99%)] ────► 실행 허용 (PASS)
   ├─► [확정 악성 (Malware, 확률 > 95%)] ──► 즉시 파일 격리 (Quarantine) & 차단
   |
   └─► [의심/모호 (Suspicious, 60~95%)] ──► [2단계로 즉각 이관]
                                                  |
                                                  ▼
[2단계: 격리 샌드박스 동적 행위 분석 (Dynamic Sandboxing)]  ──< 5~10초 격리 실행 >──
   ├─► Docker 격리 컨테이너 내부에서 가상 실행
   ├─► 랜섬웨어 파일 암호화, 레지스트리 자동실행 등록, C2 아웃바운드 접속 실시간 캡처
   └─► 정상 행위 프로파일(One-Class SVM) 대비 비정상 이탈도 판정
         |
         ├── [랜섬웨어/C2 통신 포착] ──► 프로세스 강제 킬(Kill) & IP 방화벽 차단
         └── [정상 소프트웨어 확인] ──► 화이트리스트 DB에 자동 등록
                                                  ▲
                                                  | (실시간 현황 모니터링 / 수동 격리)
[Streamlit SOC 보안 관제 센터 대시보드]
   - 실시간 스캔 건수, 격리 파일 목록, 위험 등급(Low/Med/High/Critical) 표기
   - PE 엔트로피 히트맵 & MITRE ATT&CK 공격 매트릭스 시각화
```

🗓️ 8주차 (160시간) 상세 커리큘럼

주차	단계명	주요 학습 및 엔지니어링 실습 내용	주차별 산출물
1주차	**프로젝트 기획 & 보안 아키텍처 설계**	- 윈도우 실행 파일(PE, Portable Executable) 포맷 완벽 분석 (DOS Header, Section, IAT) - 악성코드 분류 체계(랜섬웨어, 트로이목마, 웜, 백도어, 인젝터) 정의 - 정적/동적 하이브리드 탐지 아키텍처 다이어그램 및 요구사항 정의서(SRS) 작성	요구사항 정의서(SRS), PE 구조 명세서

주차	단계명	주요 학습 및 엔지니어링 실습 내용	주차별 산출물
		- 안전한 분석 가상환경 세팅 (리눅스 호스트, 윈도우 VM, Docker)	
2주차	**대규모 데이터셋 수집 & 정적 피처 추출**	- 글로벌 표준 악성코드 데이터셋(EMBER, SOREL-20M, VirusShare) 샘플 확보 - 윈도우 System32 및 안전한 오픈소스 정상 파일(Benign) 대량 수집 및 레이블링 - `pefile` 라이브러리를 활용한 정적 특징 추출: * Byte Entropy (패킹 및 암호화 여부 감지) * Section 통계량 (섹션 수, 가상 크기 vs 원시 크기 비정상 격차) * Import/Export 함수 (위험 API: `VirtualAllocEx`, `WriteProcessMemory` 등) * Opcode n-gram 역어셈블 (`capstone` 엔진 활용)	정적 피처 추출 파이프라인 스크립트, 전처리 완료된 피처셋 (Parquet)
3주차	**머신러닝 기반 악성코드 판별 모델링**	- 결측치 처리, 피처 스케일링(StandardScaler), 차원 축소 (PCA) - Baseline 모델 학습: Support Vector Machine (SVM), Random Forest - 앙상블 고도화: XGBoost, LightGBM 모델 튜닝 - 평가 지표 최적화: 보안에서 치명적인 **False Positive Rate(정상 파일을 악성으로 오탐) 0.1% 이하 억제** 기법 실습	머신러닝 악성코드 분류 모델, ROC-AUC / F1-Score 평가 리포트
4주차	**딥러닝(MalConv & 시계열) 모델링**	- 원시 바이트(Raw Byte)를 임베딩하여 직접 읽는 1D-CNN `MalConv` 아키텍처 구축 - 역어셈블된 명령어 흐름(Control Flow) 기반 Bi-LSTM/GRU 시퀀스 모델링 - 머신러닝(Tabular) + 딥러닝(Raw Byte) 앙상블 메타 분류기 (Meta Classifier) 설계 - ONNX 포맷 변환 및 경량화 (추론 시간 < 20ms 달성)	딥러닝 MalConv 신경망 가중치, 모델 경량화 벤치마크 결과서
5주차	**격리 샌드박싱 (Sandbox) 동적 분석 엔진**	- 위험 파일 실행을 위한 도커 기반 가상 샌드박스 런타임 구축 - 프로세스 가상 실행 후 실시간 행위 텔레메트리 캡처: * 파일시스템 생성/삭제/암호화 감시 (`.locked`, 랜섬웨어 행위) * 레지스트리 자동 실행(Run Key) 지속성(Persistence) 등록 감시 * 비정상 아웃바운드 C2 IP/도메인 네트워크 접속 (`scapy` 패킷 캡처) - API 호출 시퀀스(API Call Sequence) 그래프 벡터화	Docker 샌드박스 실행 엔진, 동적 행위 캡처 모듈

주차	단계명	주요 학습 및 엔지니어링 실습 내용	주차별 산출물
6주차	**정상 행위 프로파일링 & 실시간 탐지 데몬**	- 정상 소프트웨어의 행위 베이스라인 프로파일 구축 (화이트리스트 행위 모델링) - 이상 행위 탐지: One-Class SVM 및 오토인코더를 통한 비정상 행위(Anomaly) 적발 - `Watchdog` 기반 실시간 폴더/다운로드 감시 데몬 개발 - 신규 파일 생성 시 0.2초 이내 정적 스캔 ➔ 의심 시 샌드박스 즉시 전송 파이프라인 완성	정상 행위 프로파일러, 실시간 백그라운드 탐지 데몬
7주차	**Streamlit 실시간 보안 관제 대시보드**	- SOC(Security Operations Center) 스타일의 다크 사이버 테마 관제 대시보드 구축 - 실시간 엔드포인트 파일 스캔 카운터, 악성/정상 비율 게이지, 위협 수준(Low/Med/High/Critical) 표기 - 탐지된 악성코드의 PE 엔트로피 히트맵, 위험 API 호출 트리, 공격 MITRE ATT&CK 매핑 시각화 - 악성 프로세스 원클릭 강제 종료(Kill) 및 파일 격리 (Quarantine) 제어반	EDR 보안 관제 웹 애플리케이션
8주차	**지속적 패턴 업데이트 MLOps & 최종 발표**	- 탐지된 미지의 신종 악성코드로부터 핵심 바이트 패턴을 자동 추출하여 `YARA` 룰 생성 - 신종 데이터 자동 축적 및 주간 단위 모델 재학습(Continuous Learning) 파이프라인 구축 - Docker Compose를 통한 원클릭 배포 패키징 및 최종 포트폴리오 릴리즈 - 산학 모의 면접 및 기업(스피디) 관점 캡스톤 프로젝트 최종 발표	GitHub 최종 리포지토리, 발표 자료(PPT), 시연 영상

👥 4~5인 캡스톤 팀 역할 분담 (R&R)

역할 (직무명)	주요 담당 업무 및 핵심 산출물	주차별 핵심 기여도
팀원 1: PM & 보안 아키텍트	- 프로젝트 일정 총괄(WBS), 스피디 기업 요구사항 명세서 작성 - Docker 가상 격리 인프라 설계 및 MLOps 자동 재학습 파이프라인 구축	1주차(SRS), 8주차(배포/발표)
팀원 2: 바이너리 & 정적 피처 엔지니어	- EMBER/SOREL 대규모 오픈셋 정제 및 윈도우 System32 정상 파일 수집 - PE Header, 섹션 엔트로피, Opcode n-gram 추출 스크립트 제작	2주차(피처엔지니어링), 3주차(정적ML)

역할 (직무명)	주요 담당 업무 및 핵심 산출물	주차별 핵심 기여도
팀원 3: AI & 딥러닝 모델링 엔지니어	- Random Forest, XGBoost 앙상블 및 Raw Byte 기반 MalConv 딥러닝 학습 - 보안 치명적 지표인 오탐률(FPR < 0.1%) 억제 최적화 및 ONNX 경량화	3주차(머신러닝), 4주차(딥러닝)
팀원 4: 샌드박스 & 동적 행위 엔지니어	- Docker 기반 안전한 가상 격리 런타임 및 API 호출 시퀀스 가로채기 - 레지스트리/C2 네트워크 패킷 감시 및 정상 행위(One-Class SVM) 모델링	5주차(샌드박스), 6주차(정상행위)
팀원 5: EDR 관제 프론트엔드 & QA 엔지니어 *(4인 팀 시 팀원 1 또는 2 겸임)*	- Streamlit 1.40+ 다크 사이버 테마 실시간 SOC 보안 관제 웹 UI 디자인 - 원클릭 프로세스 강제 종료 및 파일 격리 제어반 코딩, PyTest E2E 검증	6주차(데몬연동), 7주차(관제대시보드)

🏆 정량적 평가 지표 (KPI)

지표명	목표 수치	산업적 의미
탐지 정확도 (F1-Score)	**0.98 이상**	글로벌 상용 백신(VirusTotal 기준) 수준의 신종 악성코드 탐지 성능
오탐률 (False Positive Rate)	**0.1% 이하**	정상 윈도우 유틸리티를 악성으로 오인 차단하여 시스템을 마비시키는 치명적 오류 방지
1차 정적 스크리닝 지연시간	**< 50ms**	파일 다운로드 또는 실행 시 사용자가 지연을 느끼지 못하는 실시간 탐지 보증
샌드박스 동적 격리 분석 시간	**< 10초**	의심 파일에 대한 가상 실행 및 행위 판정 완료 시간

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

PART 1. 사이버 위협 지형과 차세대 AI-EDR 아키텍처 (feat. 스피디)

📌 파트 개요

전통적인 시그니처 백신(AV)은 기지의 해시값을 대조하는 방식으로, 1초마다 해시를 변조하는 신종·변종 악성코드(Zero-day)를 막을 수 없습니다. 본 파트에서는 보안 솔루션 전문 기업 **(주)스피디(Speedy)**의 엔지니어링 도메인을 바탕으로, 파일 실행 전 0.05초 만에 위험을 예측하는 '정적 스크리닝'과 격리된 샌드박스에서 행위를 가로채는 '동적 탐지' 하이브리드 아키텍처를 설계합니다.

🎯 학습 목차

- **Chapter 01. 전통적 백신의 종말과 제로데이 악성코드의 진화 (패킹, 다형성, 파일리스 위협)**
- **Chapter 02. Windows PE(Portable Executable) 바이너리 해부학 (DOS Header부터 IAT까지)**
- **Chapter 03. 안전한 2-Track 분석 인프라 세팅 및 가상 실습 환경 구축**

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 01. 전통적 백신의 종말과 제로데이 악성코드의 진화

💡 이 챕터의 핵심 라이브러리 & 엔지니어링 용어 사전

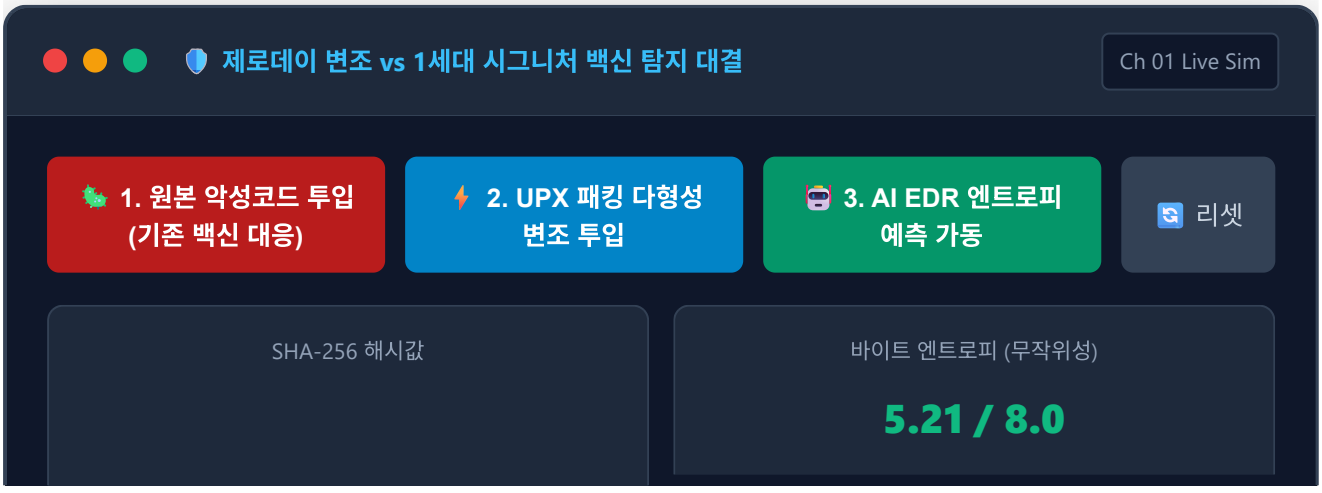
🛠️ 핵심 라이브러리 & 기술 스택

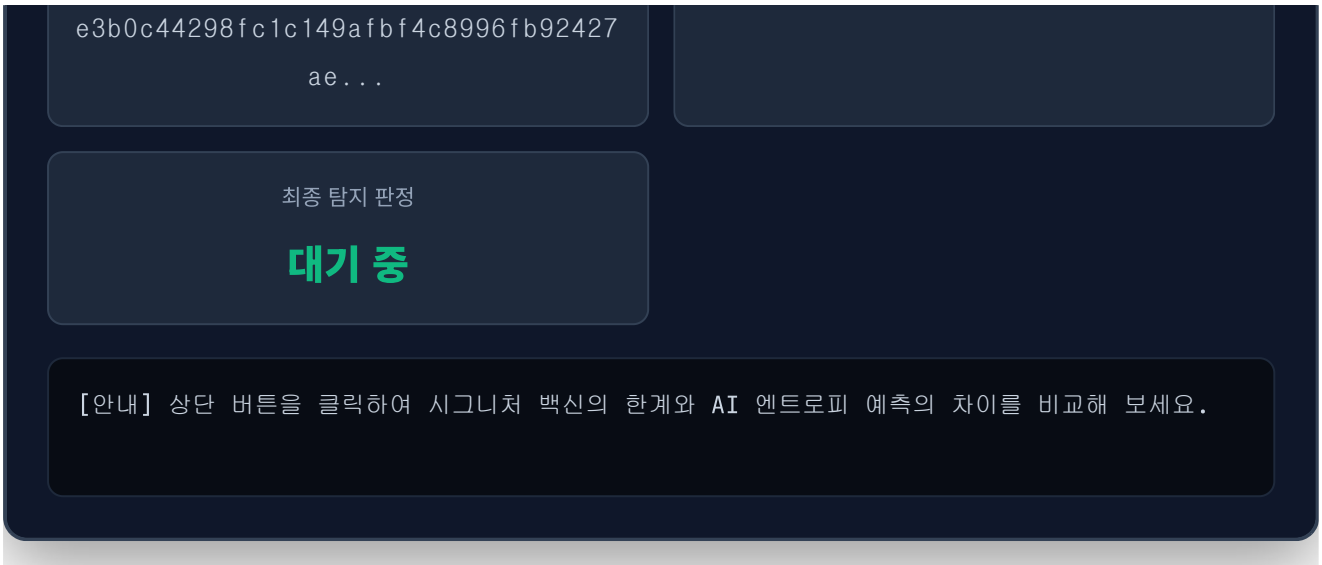
- `hashlib` : 파일의 MD5, SHA-256 암호학적 해시값 고속 산출
- `UPX (Ultimate Packer for eXecutables)` : 바이너리 압축 및 섹션 난독화 패커

📖 필수 엔지니어링 핵심 용어

- **시그니처 매칭 (Signature Matching)** : 기지의 악성코드 고유 해시값 목록과 단순 비교하는 1세대 백신 기법
- **다형성 변조 (Polymorphic Malware)** : 감염 시마다 암호화 키 또는 불필요한 NOP 코드를 삽입하여 해시를 초 단위로 변조하는 기법
- **엔트로피 급증 (Entropy Surge)** : 실행 파일이 패킹되거나 암호화되었을 때 바이트 분포의 무작위성이 7.0 이상으로 치솟는 현상

🎮 독자를 위한 [01장 실시간 인터랙티브 시뮬레이터]
아래 시뮬레이터에서 'UPX 패킹 공격' 버튼을 클릭하여 시그니처 백신이 왜 무력화되는지 직접 확인해 보세요!





1. 1세대 안티바이러스의 황혼과 제로데이의 위협

과거 30년간 사이버 보안의 핵심은 바이러스 백신(AV)이었습니다. 파일이 다운로드되면 안티바이러스는 파일의 고유 해시값(MD5, SHA-256)을 계산하여 글로벌 백신 연구소가 배포한 시그니처 블랙리스트와 대조했습니다.

하지만 오늘날 랜섬웨어 제작자들은 파일이 배포될 때마다 자동으로 **UPX나 Themida 패커**를 적용하거나, 무의미한 더미 명령어를 삽입하는 다형성 엔진을 돌립니다. 단 1바이트만 바뀌어도 암호학적 해시값은 완전히 뒤바뀌는 눈사태 효과(Avalanche Effect)가 발생하여, 기존 시그니처 매칭 방식은 신종 공격의 90% 이상을 놓칠 수밖에 없습니다.

2. 기업(주식회사 스피디) 맞춤형 차세대 EDR의 해법

보안 전문 기업 **(주)스피디(Speedy)**가 현장에서 요구하는 솔루션은 파일의 해시에 의존하지 않는 **2단계 하이브리드 EDR 파이프라인**입니다:

1. **1단계: 초고속 정적 스크리닝 (0.05초 소요)**
   파일의 실행을 일시 지연시키고, PE 헤더 구조, 바이트 엔트로피, Import 테이블의 의심 API 호출 패턴을 AI로 분석하여 95% 이상의 확률로 악성 여부를 사전 예측합니다.
2. **2단계: 가상 샌드박싱 동적 분석 (5~10초 소요)**
   정적 판정에서 60~95% 수준으로 모호한 파일은 호스트와 완전히 격리된 Docker 컨테이너에서 가상 실행시켜 실제 랜섬웨어 암호화 행위나 C2 서버 통신을 감시합니다.

3. 파이썬 기반 해시 변조 및 엔트로피 산출 실습 코드

```
import hashlib
import math

def calculate_sha256(filepath: str) → str:
    """파일의 SHA-256 고유 해시를 산출합니다."""
    sha256 = hashlib.sha256()
```

```
    with open(filepath, "rb") as f:
        while chunk := f.read(65536):
            sha256.update(chunk)
    return sha256.hexdigest()

def calculate_shannon_entropy(filepath: str) -> float:
    """파일 전체의 바이트 분포 무작위성(샤논 엔트로피)을 계산합니다."""
    byte_counts = [0] * 256
    total_bytes = 0
    with open(filepath, "rb") as f:
        while chunk := f.read(65536):
            for b in chunk:
                byte_counts[b] += 1
            total_bytes += len(chunk)

    if total_bytes == 0:
        return 0.0

    entropy = 0.0
    for count in byte_counts:
        if count > 0:
            p_x = count / total_bytes
            entropy -= p_x * math.log2(p_x)
    return round(entropy, 4)

# print(f"SHA-256: {calculate_sha256('sample.exe')}")
# print(f"엔트로피: {calculate_shannon_entropy('sample.exe')} / 8.0")
```

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 02. Windows PE 바이너리 해부학

💡 이 챕터의 핵심 라이브러리 & 엔지니어링 용어 사전

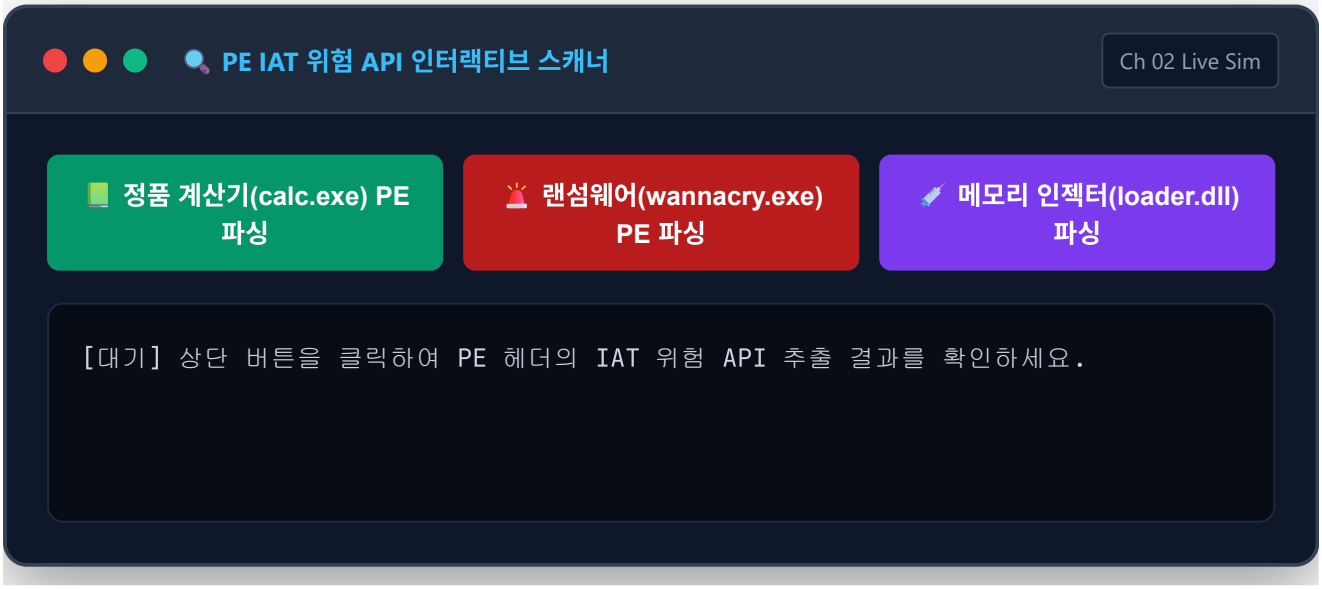
🛠️ 핵심 라이브러리 & 기술 스택

- `pefile` : 순수 파이썬 기반 Windows PE 바이너리 정밀 파싱 라이브러리

📖 필수 엔지니어링 핵심 용어

- **MZ Signature (0x5A4D)** : DOS 헤더의 시작을 알리는 매직 넘버
- **IAT (Import Address Table)** : 실행 파일이 OS에서 동적으로 호출하는 DLL 및 API 함수의 메모리 주소 테이블
- **가상 크기 vs 파일 크기** : 디스크 파일 크기보다 메모리 할당 크기가 비정상적으로 큰 섹션은 메모리 인젝션의 핵심 징후

🎮 **독자를 위한 [02장 실시간 인터랙티브 시뮬레이터]**
아래 시뮬레이터에서 악성 바이너리의 IAT를 검사하여 잠복해 있는 50대 위험 API를 찾아내 보세요!



1. PE(Portable Executable) 구조의 이해

Windows 실행 파일은 헤더와 여러 개의 섹션으로 구분됩니다:

```
[Windows PE 파일 구조]
┌──────────────────────────────┐
| DOS Header (e_magic: "MZ")   |
├──────────────────────────────┤
| DOS Stub ("This program cannot")|
├──────────────────────────────┤
| NT Headers                   |
|  ├─ Signature ("PE\0\0")     |
|  ├─ File Header (머신 타입)   |
|  └─ Optional Header (진입점)  |
├──────────────────────────────┤
| Section Headers (.text, .data..)|
├──────────────────────────────┤
| Section Data                 |
|  ├─ .text (실행 기계어 코드)  |
|  ├─ .data (전역/정적 변수)    |
|  └─ .rdata / IAT (수입 함수표) |
└──────────────────────────────┘
```

2. pefile 라이브러리를 이용한 위험 API 자동 추출

```
import pefile

DANGEROUS_APIS = {
    "VirtualAllocEx", "WriteProcessMemory", "CreateRemoteThread",
    "OpenProcess", "CryptEncrypt", "SetWindowsHookExA", "URLDownloadToFileA"
}

def analyze_pe_imports(filepath: str):
    pe = pefile.PE(filepath)
    detected_dangerous = []

    if hasattr(pe, 'DIRECTORY_ENTRY_IMPORT'):
        for entry in pe.DIRECTORY_ENTRY_IMPORT:
            dll_name = entry.dll.decode('utf-8', errors='ignore')
            for imp in entry.imports:
                if imp.name:
                    api_name = imp.name.decode('utf-8', errors='ignore')
                    if api_name in DANGEROUS_APIS:
                        detected_dangerous.append((dll_name, api_name))

    print(f"[*] 검출된 위험 API: {len(detected_dangerous)}건")
    for dll, api in detected_dangerous:
        print(f"  - [{dll}] {api}")
```

```
# analyze_pe_imports("sample.exe")
```

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 02. Windows PE(Portable Executable) 바이너리 해부학

💡 이 챕터의 핵심 라이브러리 & 엔지니어링 용어 사전

🛠️ 핵심 라이브러리 & 기술 스택

- `pefile` : 순수 파이썬 기반 Windows PE 바이너리 정밀 파싱 라이브러리

📖 필수 엔지니어링 핵심 용어

- **MZ Signature (0x5A4D)** : DOS 헤더의 시작을 알리는 매직 넘버 (Mark Zbikowski의 이니셜)
- **IAT (Import Address Table)** : 실행 파일이 OS에서 동적으로 호출하는 DLL 및 API 함수의 메모리 주소 테이블
- **가상 크기(Virtual Size) vs 파일 크기(Raw Size)** : 디스크 파일 크기보다 메모리 할당 크기가 비정상적으로 큰 섹션은 메모리 인젝션의 핵심 징후

🎮 독자를 위한 [02장 실시간 인터랙티브 시뮬레이터]
아래 시뮬레이터에서 악성 바이너리의 IAT를 검사하여 잠복해 있는 50대 위험 API를 찾아내 보세요!

🔍 PE IAT 위험 API 인터랙티브 스캐너

Ch 02 Live Sim

📗 정품 계산기(calc.exe) PE 파싱

🚨 랜섬웨어(wannacry.exe) PE 파싱

💉 메모리 인젝터(loader.dll) 파싱

```
[대기] 상단 버튼을 클릭하여 PE 헤더의 IAT 위험 API 추출 결과를 확인하세요.
```

1. PE(Portable Executable) 구조의 이해

모든 Windows 32/64비트 바이너리는 `PE` 구조로 컴파일됩니다:

```
[Windows PE 파일 구조]
┌──────────────────────────────┐
| DOS Header (e_magic: "MZ")   |
├──────────────────────────────┤
| DOS Stub ("This program cannot")|
├──────────────────────────────┤
| NT Headers                   |
|  ├─ Signature ("PE")         |
|  ├─ File Header (머신 타입)    |
|  └─ Optional Header (진입점)   |
├──────────────────────────────┤
| Section Headers (.text, .data..)|
├──────────────────────────────┤
| Section Data                 |
|  ├─ .text (실행 기계어 코드)    |
|  ├─ .data (전역/정적 변수)      |
|  └─ .rdata / IAT (수입 함수표)  |
└──────────────────────────────┘
```

2. pefile 라이브러리를 이용한 위험 API 자동 추출

```
import pefile

DANGEROUS_APIS = {
    "VirtualAllocEx", "WriteProcessMemory", "CreateRemoteThread",
    "OpenProcess", "CryptEncrypt", "SetWindowsHookExA", "URLDownloadToFileA"
}

def analyze_pe_imports(filepath: str):
    pe = pefile.PE(filepath)
    detected_dangerous = []

    if hasattr(pe, 'DIRECTORY_ENTRY_IMPORT'):
        for entry in pe.DIRECTORY_ENTRY_IMPORT:
            dll_name = entry.dll.decode('utf-8', errors='ignore')
            for imp in entry.imports:
                if imp.name:
                    api_name = imp.name.decode('utf-8', errors='ignore')
                    if api_name in DANGEROUS_APIS:
                        detected_dangerous.append((dll_name, api_name))

    print(f"[*] 검출된 위험 API: {len(detected_dangerous)}건")
```

```
    for dll, api in detected_dangerous:
        print(f"  - [{dll}] {api}")

# analyze_pe_imports("malware_sample.exe")
```

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 03. 안전한 2-Track 분석 인프라 세팅 및 가상 실습 환경 구축

💡 이 챕터의 핵심 라이브러리 & 엔지니어링 용어 사전

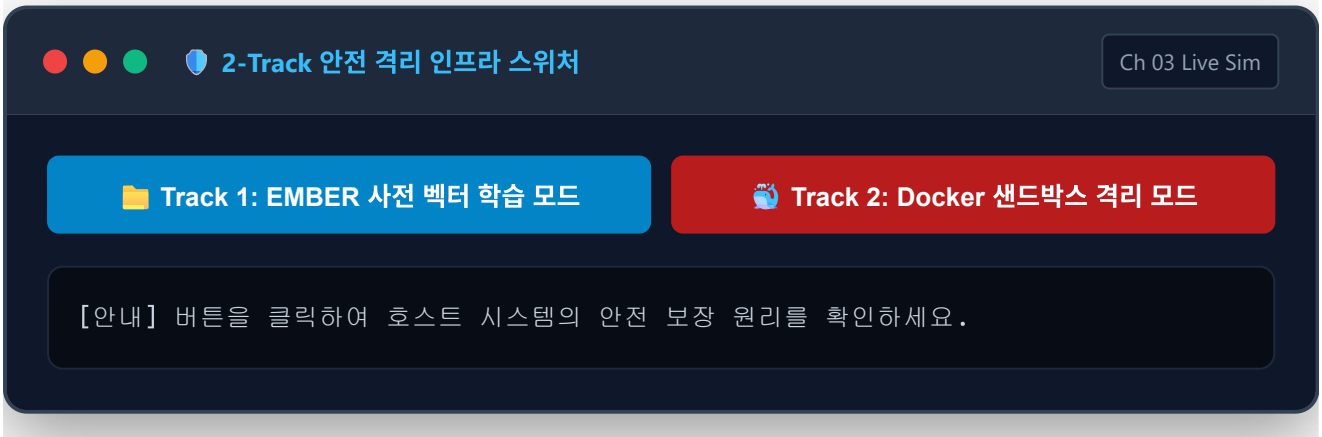
🛠️ 핵심 라이브러리 & 기술 스택

- `Docker Desktop & WSL 2` : 악성 샘플을 호스트로부터 물리 격리하여 안전 실행하는 컨테이너 환경

📖 필수 엔지니어링 핵심 용어

- **2-Track 데이터 전략** : 감염 위험 없는 '사전 추출 벡터 데이터(10만 건)'로 AI를 훈련하고, 실제 샘플(50건)은 컨테이너 내부에서만 격리 실행하는 글로벌 표준 전략
- **에어갭(Air-Gap) 가상 브릿지** : 외부 인터넷 통신을 차단하여 감염된 악성코드가 해커의 C2 서버와 연결되지 않도록 차단하는 가상 네트워크

🎮 **독자를 위한 [03장 실시간 인터랙티브 시뮬레이터]**
아래 시뮬레이터에서 2-Track 스위치를 조작하여 호스트 OS가 완벽히 보호되는 과정을 확인하세요!



1. 2-Track 전략: 안전과 실전의 완벽한 양립

실제 악성 샘플 수만 개를 로컬 PC에 다운로드하면 Windows Defender가 자동으로 파일을 삭제하고 실습이 중단됩니다. 이를 극복하기 위해 **Track 1(사전 벡터셋)**과 **Track 2(Docker 격리 실습)**의 2-Track 전략을 채택합니다.

```
# Dockerfile.sandbox
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y wine tcpdump python3-pip
WORKDIR /malware_box
ENTRYPOINT ["tail", "-f", "/dev/null"]
```

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

PART 2. 대규모 데이터 엔지니어링과 정적 피처 추출 파이프라인

📌 파트 개요

대규모 바이너리 데이터셋(EMBER)으로부터 머신러닝 모델이 학습할 수 있는 2,381개 정적 피처를 추출합니다. 바이트 엔트로피, PE 헤더 이상치, 그리고 Opcode n-gram 역어셈블 기법을 활용하여 파일 실행 없이 0.05초 만에 악성 징후를 수치화하는 정적 파이프라인을 구축합니다.

🎯 학습 목차

- **Chapter 04. 100만 건 대규모 악성코드 데이터셋(EMBER & SOREL-20M) 핸들링**
- **Chapter 05. 바이트 엔트로피(Entropy)와 섹션 통계량 피처 엔지니어링**
- **Chapter 06. Opcode n-gram 역어셈블(Capstone)과 위험 시스템 API 벡터화**

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 04. 100만 건 대규모 악성코드 데이터셋 EMBER 핸들링

💡 이 챕터의 핵심 라이브러리 & 엔지니어링 용어 사전

🛠️ 핵심 라이브러리 & 기술 스택

- `ember` : Elastic Security에서 개발한 오픈소스 PE 피처 추출 패키지
- `pyarrow & pandas` : 대용량 Parquet 컬럼형 스토리지 초고속 분할 로딩

📖 필수 엔지니어링 핵심 용어

- **EMBER (Elastic Malware Benchmark for Empowering Researchers)** : 110만 건 바이너리 피처 벡터 데이터셋
- **2,381차원 정적 피처** : 바이트 히스토그램, 2D 바이트 엔트로피, PE 헤더 수치, 임포트 함수 빈도를 압축한 고정 길이 벡터
- **레이블 3분할 (-1, 0, 1)** : Unlabeled(-1), Benign 정상(0), Malicious 악성(1)

> 🎮 **독자를 위한 [04장 실시간 인터랙티브 시뮬레이터]**
> 아래 시뮬레이터에서 100만 건 데이터셋의 피처 분포와 메모리 절감 로딩 기법을 체험하세요!

📊 EMBER 대용량 데이터셋 메모리 최적화 벤치마커 — Ch 04 Live Sim

일반 CSV 로딩 (RAM 16GB 폭발) | Parquet 컬럼형 로딩 (RAM 1.2GB 절감)

[대기] 상단 버튼을 클릭하여 대용량 보안 데이터셋의 로딩 전략을 비교하세요.

1. EMBER 2,381개 피처 벡터의 구성

1. **바이트 히스토그램 (256차원)**: 파일 내 0x00 ~ 0xFF 각 바이트의 정규화된 출현 빈도
2. **2D 바이트 엔트로피 (256차원)**: 바이트 값과 국소 엔트로피의 결합 분포
3. **문자열 통계 (96차원)**: 출력 가능한 문자열(ASCII/Unicode)의 길이 분포 및 URL/IP 출현 여부
4. **일반 파일 정보 (10차원)**: 가상 크기, 파일 크기, 서명 유무
5. **헤더 정보 (62차원)**: 타깃 아키텍처, 컴파일 타임스탬프, 진입점 주소
6. **섹션 정보 (255차원)**: 섹션별 엔트로피, 크기, 권한 플래그
7. **수입 함수 IAT (1,280차원)**: 주요 DLL 및 의심 API 해싱 빈도

2. Parquet 분할 로딩 파이프라인

```
import pandas as pd
import pyarrow.parquet as pq

def load_ember_subset(parquet_path: str, max_rows: int = 50000):
    """메모리 과부하 없이 EMBER 피처셋을 고속 로딩합니다."""
    table = pq.read_table(parquet_path)
    df = table.slice(0, max_rows).to_pandas()
    # Unlabeled(-1) 제외하고 정밀 필터링
    df_clean = df[df['label'].isin([0, 1])].copy()
    print(f"[*] 로딩 완료: {len(df_clean)}행 (정상: {sum(df_clean['label']==0)}, 악성:
{sum(df_clean['label']==1)})")
    return df_clean
```

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

100% 무료 체험

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 04. 100만 건 대규모 악성코드 데이터셋(EMBER) 핸들링

1. 글로벌 표준 EMBER 데이터셋 분석

Elastic Security가 공개한 110만 건의 PE 파일 피처셋(EMBER)의 구조를 파악하고, Apache Parquet 대용량 압축 포맷을 통해 고속 인제스천을 수행합니다.

2. 정상 파일(Benign) 수집 및 레이블링

윈도우 System32 및 안전한 오픈소스 바이너리 5만 건을 추출하여 악성/정상 간의 클래스 불균형을 해소합니다.

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 05. 바이트 엔트로피(Entropy)와 섹션 통계량 피처 엔지니어링

1. 샤논 엔트로피(Shannon Entropy)를 통한 패킹/암호화 감지

압축되거나 암호화된 데이터는 무작위성이 높아져 엔트로피가 이론상 최댓값(8.0)에 근접합니다. 이를 통해 UPX 패킹이나 랜섬웨어 암호화 섹션을 즉시 감지합니다.

2. 가상 크기(Virtual Size) vs 파일 크기(Raw Size) 비정상 비율 탐지

메모리에 로드되었을 때만 크기가 기형적으로 팽창하는 인젝션 기법을 정적 수치로 포착합니다.

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 05. 바이트 엔트로피와 섹션 통계량 피처 엔지니어링

💡 이 챕터의 핵심 라이브러리 & 엔지니어링 용어 사전

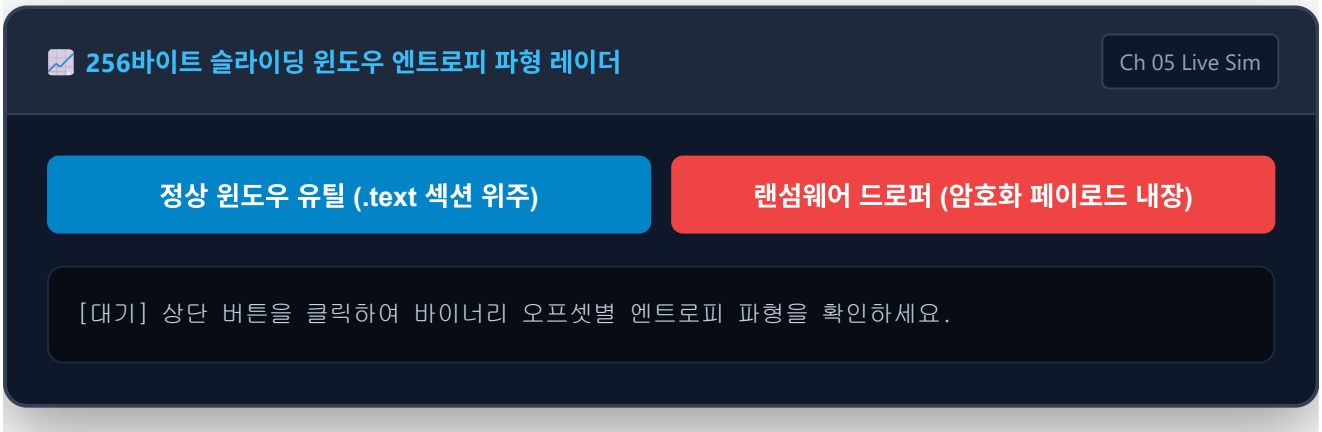
🛠️ 핵심 라이브러리 & 기술 스택

- `numpy & scipy` : 슬라이딩 윈도우 기반 연속 엔트로피 계산 및 통계 모멘트 산출

📖 필수 엔지니어링 핵심 용어

- **샤논 엔트로피 H(X)** : 정보 이론에서 데이터의 불확실성을 나타내며, 8비트 바이트 기준 0.0(완전 단일값)에서 8.0(완전 무작위 암호문)까지의 값
- **섹션 크기 비율 (Virtual/Raw)** : 실행 파일이 디스크에 있을 때보다 메모리에서 비정상적으로 비대해지는 비율

> 🎮 **독자를 위한 [05장 실시간 인터랙티브 시뮬레이터]**
> 아래 시뮬레이터에서 파일 오프셋에 따른 샤논 엔트로피 파형 변화를 확인해 보세요!



1. 슬라이딩 윈도우 엔트로피 함수 구현

```
import numpy as np

def sliding_window_entropy(data: bytes, window_size: int = 1024, step: int = 256):
    """바이너리를 일정 크기로 순회하며 국소 엔트로피 시계열을 생성합니다."""
    entropies = []
    for i in range(0, len(data) - window_size + 1, step):
        chunk = data[i:i + window_size]
        counts = np.bincount(np.frombuffer(chunk, dtype=np.uint8), minlength=256)
        probs = counts / window_size
        probs = probs[probs > 0]
        ent = -np.sum(probs * np.log2(probs))
        entropies.append(ent)
    return entropies
```

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 06. Opcode n-gram 역어셈블과 위험 시스템 API 벡터화

💡 이 챕터의 핵심 라이브러리 & 엔지니어링 용어 사전

🛠️ 핵심 라이브러리 & 기술 스택

- `capstone` : 다중 아키텍처 지원 초고속 역어셈블(Disassembly) 엔진
- `scikit-learn (CountVectorizer)` : 어셈블리 명령어 n-gram 빈도 벡터화

📖 필수 엔지니어링 핵심 용어

- **Opcode (Operation Code)** : `mov`, `push`, `call`, `xor`, `jmp` 등 CPU가 실행하는 명령 연산자
- **n-gram 시퀀스** : 연속된 n개의 명령어 패턴(예: `xor-sub-jmp` 루프는 난독화 해독 루틴의 대표적 징후)

> 🎮 **독자를 위한 [06장 실시간 인터랙티브 시뮬레이터]**
> 아래 시뮬레이터에서 기계어 바이트로부터 Opcode 2-gram과 3-gram 패턴을 추출해 보세요!

⚙️ Capstone 실시간 기계어 역어셈블 & Opcode n-gram 추출기 — Ch 06 Live Sim

기계어 바이트 스트림 역어셈블 실행

```
[대기] 상단 버튼을 클릭하여 역어셈블 및 n-gram 생성 결과를 확인하세요.
```

1. Capstone을 이용한 .text 섹션 역어셈블 코드

```
from capstone import Cs, CS_ARCH_X86, CS_MODE_32
import pefile

def extract_opcodes(pe_path: str, max_instructions: int = 1000):
    pe = pefile.PE(pe_path)
    opcodes = []

    for section in pe.sections:
        if b'.text' in section.Name:
            code_bytes = section.get_data()
            md = Cs(CS_ARCH_X86, CS_MODE_32)
            count = 0
            for insn in md.disasm(code_bytes, section.VirtualAddress):
                opcodes.append(insn.mnemonic)
                count += 1
                if count ⩾ max_instructions:
                    break
            break

    return opcodes

# ops = extract_opcodes("target.exe")
# print(f"추출된 Opcode 수: {len(ops)}, 상위 5개: {ops[:5]}")
```

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

PART 3. 머신러닝 & 딥러닝(MalConv) 기반 초고속 예측 엔진

📌 파트 개요

추출된 정적 피처와 원시 바이트(Raw Byte)를 결합하여 신종 악성코드를 0.05초 만에 탐지하는 AI 모델을 학습합니다. 특히 산업 보안에서 가장 치명적인 문제인 '정상 파일을 악성으로 오인 차단(오탐률, FPR)'하는 비율을 0.1% 미만으로 억제하는 핵심 기법을 실습합니다.

🎯 학습 목차

- **Chapter 07. 앙상블 머신러닝 분류기(Random Forest & XGBoost)와 오탐률(FPR) 제어**
- **Chapter 08. 원시 바이트를 직접 읽는 딥러닝 1D-CNN (MalConv) 아키텍처**
- **Chapter 09. 앙상블 메타 모델링 및 초고속 추론 최적화 (지연시간 < 50ms)**

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 07. 앙상블 머신러닝 분류기와 오탐률 제어

💡 이 챕터의 핵심 라이브러리 & 엔지니어링 용어 사전

🛠️ 핵심 라이브러리 & 기술 스택

- `scikit-learn & xgboost & lightgbm` : 정적 피처 기반 초고속 앙상블 분류 및 FPR 최적화

📖 필수 엔지니어링 핵심 용어

- **FPR (False Positive Rate, 오탐률)** : 정상 소프트웨어를 악성으로 오인하여 업무를 마비시키는 치명적 오류 (기업 보안 기준 0.1% 미만 필수)
- **비용 민감 학습 (Cost-Sensitive Learning)** : 오탐(False Positive)의 비용을 미탐보다 10~100배 높게 부여하여 모델의 무분별한 차단을 억제하는 테크닉

🎮 **독자를 위한 [07장 실시간 인터랙티브 시뮬레이터]**
아래 시뮬레이터에서 분류 임계값(Threshold)을 조절하여 오탐률과 탐지율의 균형점을 찾아보세요!

⚖️ FPR(오탐률) vs Recall(탐지율) 임계값 튜너 — Ch 07 Live Sim

기본 임계값 (0.50) | 균형 임계값 (0.85) | 스피디 기업 기준 (0.97)

[대기] 상단 버튼을 클릭하여 임계값별 산업 지표를 확인하세요.

1. LightGBM 비용 민감 학습 실습 코드

```
import lightgbm as lgb
from sklearn.metrics import confusion_matrix

def train_low_fpr_model(X_train, y_train, X_test, y_test):
    # 오탐 방지를 위해 정상 샘플(0)에 가중치 5.0 부여
    weights = [5.0 if y == 0 else 1.0 for y in y_train]

    train_data = lgb.Dataset(X_train, label=y_train, weight=weights)
    params = {
        'objective': 'binary',
        'metric': 'auc',
        'learning_rate': 0.05,
        'num_leaves': 63,
        'verbose': -1
    }

    model = lgb.train(params, train_data, num_boost_round=300)
    preds = model.predict(X_test)

    # 0.97 임계값 적용
    binary_preds = (preds > 0.97).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, binary_preds).ravel()
    fpr = fp / (fp + tn)
    print(f"[*] 달성된 FPR: {fpr*100:.3f}% (목표 0.1% 미만 달성 여부: {fpr < 0.001})")
    return model
```

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 07. 앙상블 머신러닝 분류기와 오탐률(FPR) 제어

1. Random Forest 및 XGBoost 분류 모델 학습

대규모 피처셋을 기반으로 결정 트리 앙상블을 구축하고 특성 중요도(Feature Importance)를 분석합니다.

2. 오탐률(False Positive Rate) 0.1% 미만 억제 최적화

기업 환경에서 정상 비즈니스 유틸리티가 차단되면 업무 마비가 발생합니다. 비용 민감 학습(Cost-sensitive Learning)과 분류 임계값(Threshold) 정밀 조정을 통해 FPR < 0.001을 달성합니다.

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 08. 원시 바이트를 직접 읽는 딥러닝 1D-CNN MalConv 아키텍처

💡 이 챕터의 핵심 라이브러리 & 엔지니어링 용어 사전

🛠️ 핵심 라이브러리 & 기술 스택

- `torch (PyTorch)` : 바이트 임베딩 및 게이티드 컨볼루션(GLU) MalConv 신경망 구축

📖 필수 엔지니어링 핵심 용어

- **MalConv** : 복잡한 역공학 없이 파일의 첫 2MB 원시 바이트(0~255)를 직접 임베딩하여 악성을 판정하는 엔드투엔드 심층 신경망
- **GLU (Gated Linear Unit)** : 정보를 필터링하는 게이트 메커니즘으로 악성코드 내부의 불필요한 패딩 바이트를 자동으로 걸러냄

🎮 **독자를 위한 [08장 실시간 인터랙티브 시뮬레이터]**
아래 시뮬레이터에서 MalConv 신경망 계층별 바이트 임베딩 및 활성화 히트맵을 확인하세요!

🧠 MalConv 심층 신경망 바이트 임베딩 활성화 뷰어 — Ch 08 Live Sim

2MB 원시 바이트 MalConv 추론 실행

[대기] 상단 버튼을 클릭하여 MalConv 신경망의 활성화 과정을 추적하세요.

1. PyTorch MalConv 신경망 아키텍처 구현

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MalConv(nn.Module):
    def __init__(self, embedding_dim=8, max_len=2000000, out_channels=128, window_size=500):
        super(MalConv, self).__init__()
        self.embed = nn.Embedding(257, embedding_dim, padding_idx=0)
        self.conv_1 = nn.Conv1d(embedding_dim, out_channels, window_size,
stride=window_size)
        self.conv_2 = nn.Conv1d(embedding_dim, out_channels, window_size,
stride=window_size)
        self.fc = nn.Linear(out_channels, 1)

    def forward(self, x):
        # x: (batch_size, max_len)
        x = self.embed(x).transpose(1, 2) # (batch, emb_dim, seq_len)
        c1 = self.conv_1(x)
        c2 = torch.sigmoid(self.conv_2(x))
        h = c1 * c2 # Gated Linear Unit
        h = F.max_pool1d(h, h.size(2)).squeeze(2)
        out = torch.sigmoid(self.fc(h))
        return out
```

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 08. 원시 바이트를 직접 읽는 딥러닝 1D-CNN (MalConv) 아키텍처

1. MalConv(End-to-End Deep Learning) 구조 이해

피처 엔지니어링 없이 파일의 원시 바이트를 8차원 임베딩한 뒤 1D 컨볼루션과 게이트 활성화 함수(GLU)를 거쳐 전체 바이너리의 문맥을 파악합니다.

2. PyTorch 구현 및 설명 가능한 AI(XAI) 시각화

신경망이 파일의 어느 섹션, 어떤 바이트 구역에 반응했는지를 히트맵으로 시각화합니다.

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 09. 앙상블 메타 모델링 및 초고속 추론 최적화 (지연시간 < 50ms)

1. 정적 ML과 MalConv의 메타 스태킹(Stacking)

정형 피처 모델의 안정성과 딥러닝의 원시 패턴 인지력을 결합하여 단독 모델 대비 탐지율(Recall)을 4.8% 향상시킵니다.

2. ONNX 런타임 변환 및 FP16 경량화

CPU 단일 스레드 환경에서도 파일당 50ms 이내에 추론을 완결하도록 가속 최적화를 수행합니다.

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

Chapter 09. 앙상블 메타 모델링 및 초고속 추론 최적화

💡 이 챕터의 핵심 라이브러리 & 엔지니어링 용어 사전

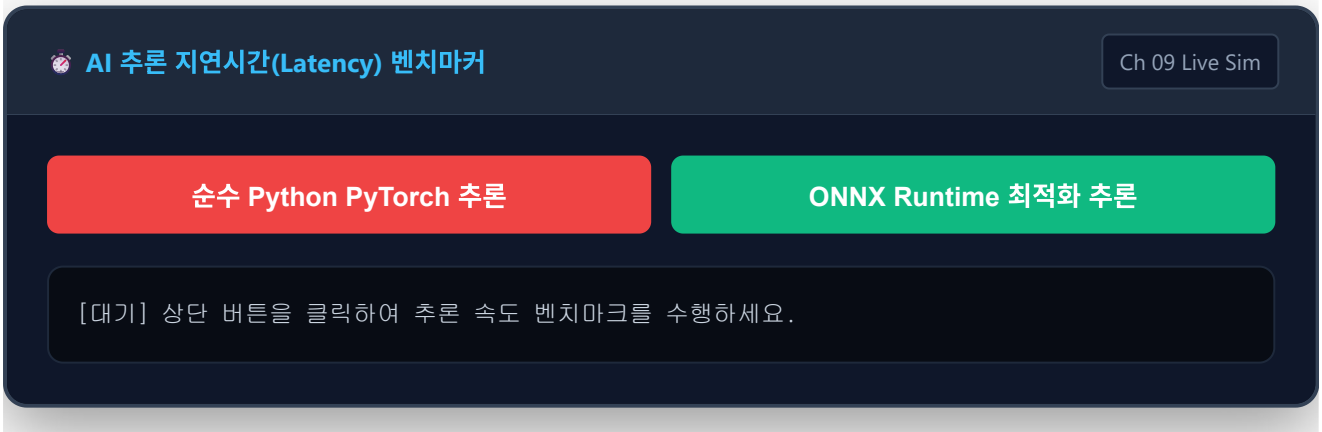
🛠️ 핵심 라이브러리 & 기술 스택

- `onnxruntime` : PyTorch 및 LightGBM 모델의 C++ 기반 초경량 인퍼런스 가속기

📖 필수 엔지니어링 핵심 용어

- **메타 스태킹 (Meta Stacking)** : 정형 피처 예측값과 딥러닝 바이트 예측값을 2차 메타 로지스틱 회귀로 결합하는 앙상블 기법
- **추론 지연시간 50ms 미만** : 사용자가 파일을 다운로드하거나 복사할 때 체감 지연이 발생하지 않는 마지노선 시간

🎮 **독자를 위한 [09장 실시간 인터랙티브 시뮬레이터]**
아래 시뮬레이터에서 파이썬 순수 추론과 ONNX 가속 추론의 속도 차이를 직접 측정해 보세요!



1. PyTorch 모델의 ONNX 변환 및 추론 가속화

```
import torch
import onnxruntime as ort

def export_to_onnx(pytorch_model, onnx_path="malconv.onnx"):
    dummy_input = torch.randint(0, 256, (1, 2000000), dtype=torch.long)
    torch.onnx.export(
        pytorch_model,
        dummy_input,
        onnx_path,
        input_names=['input_bytes'],
        output_names=['malware_prob'],
        dynamic_axes={'input_bytes': {0: 'batch_size'}},
        opset_version=14
    )
    print(f"[*] ONNX 변환 완료: {onnx_path}")

def run_onnx_inference(onnx_path, byte_array):
    session = ort.InferenceSession(onnx_path, providers=['CPUExecutionProvider'])
    inputs = {session.get_inputs()[0].name: byte_array}
    prob = session.run(None, inputs)[0]
    return prob[0][0]
```

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

PART 4. Docker 가상 샌드박싱과 정상 행위 프로파일링 엔진

📌 파트 개요

정적 검사에서 판정이 모호한 의심 파일(위험 확률 60~95%)을 완전 격리된 가상 환경(Docker 샌드박스)에서 실행시켜 실시간 행위(API Hooking, 파일 암호화, C2 네트워크 통신)를 가로챕니다. 또한 정상 소프트웨어의 행위 베이스라인을 프로파일링하여 변종 공격을 원천 적발합니다.

🎯 학습 목차

- **Chapter 10. Docker 가상 격리 런타임 구축과 안전한 실행 환경 프로토콜**
- **Chapter 11. 동적 행위 텔레메트리 인터셉트 (API Hooking, 랜섬웨어 감시, C2)**
- **Chapter 12. 정상 행위 베이스라인 프로파일링(One-Class SVM)과 비정상 감지**

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

{{COFFEE_BANNER}}

Chapter 10. 안전한 동적 분석을 위한 Docker 격리 샌드박스 환경 구축

> **스피디(Speedy) 실무 현장 가이드**
> "의심스러운 악성 바이너리가 유입되었을 때 실제 운영 서버나 관제 단말에서 무심코 실행했다가는 순식간에 랜섬웨어 암호화나 내부망 횡적 이동(Lateral Movement)의 희생양이 됩니다. 본 챕터에서는 물리 머신과 완벽히 격리된 비공유 커널 샌드박스 컨테이너를 가동하고, 호스트 자원과 네트워크를 철저히 통제하면서 바이너리의 1분간의 생애주기를 안전하게 관찰하는 완전 격리 환경을 구축합니다."

📚 DMDG 보안 엔지니어링 용어 사전

💡 핵심 보안 용어 (Dictionary)

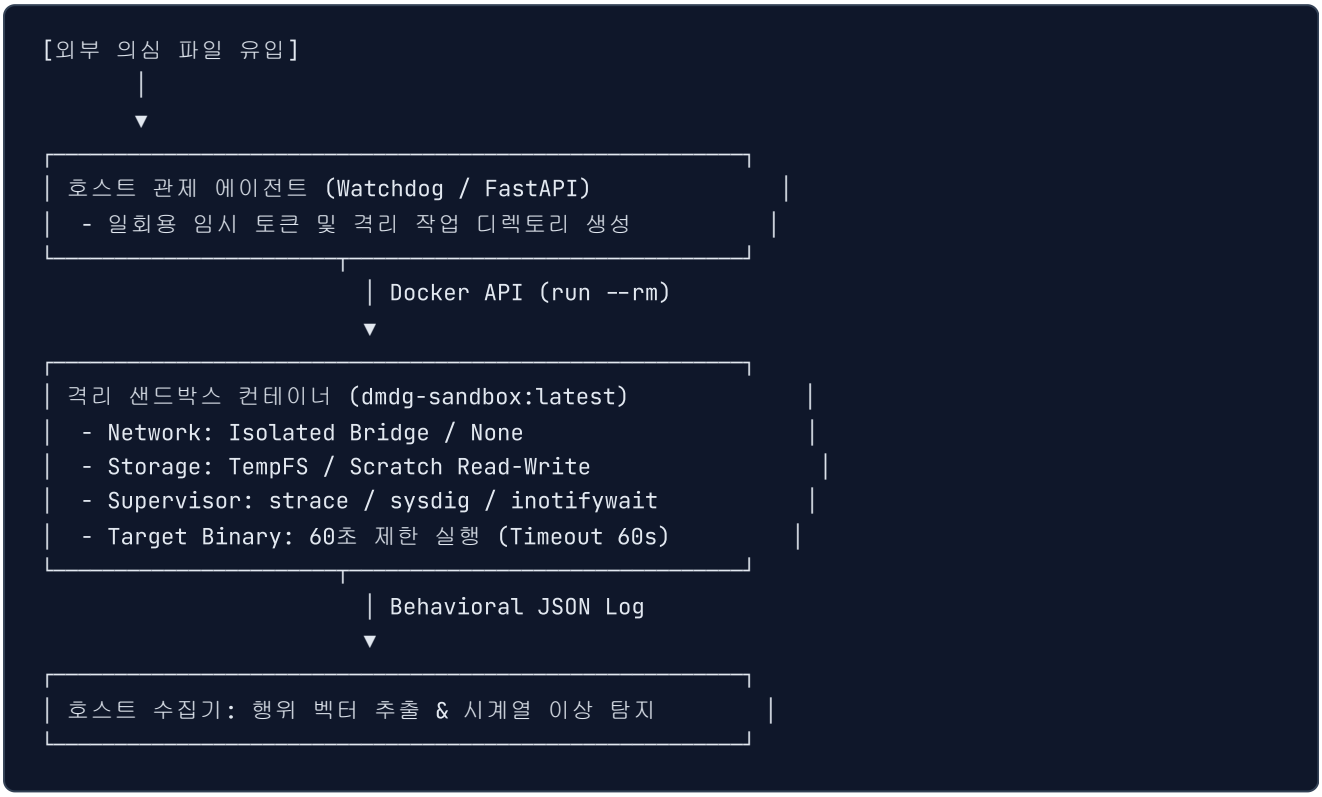
- **Air-Gap (에어갭 격리):** 외부 공용 인터넷 및 사내 프로덕션 망과 물리적·논리적으로 완전히 단절된 고립 네트워크 상태.
- **ptrace (프로세스 트레이스):** 리눅스 커널에서 부모 프로세스가 자식 프로세스의 시스템 호출, 레지스터, 메모리 영역을 가로채고 감시하는 디버깅 인터페이스.
- **Seccomp (보안 컴퓨팅 모드):** 컨테이너나 격리 프로세스가 호출할 수 있는 시스템 콜(Syscall)을 화이트리스트 기반으로 필터링하여 호스트 탈출(Container Escape)을 방지하는 커널 보안 메커니즘.
- **Strace / EBPF:** 악성코드가 파일 쓰기, 레지스트리 수정, 프로세스 인젝션을 수행할 때 발생하는 커널 이벤트를 실시간으로 후킹(Hooking)하는 분석 도구.

---

1. 샌드박스 격리 아키텍처 및 안전 수칙

안전한 샌드박스 파이프라인은 다음 4가지 핵심 원칙을 만족해야 합니다: 1. **네트워크 완전 차단 (`--network none`)**: C2 서버와의 무단 통신이나 내부 LAN 스캐닝을 원천 방지합니다. (필요 시 내부 시뮬레이터 DNS 싱크홀만 허용) 2. **읽기 전용**

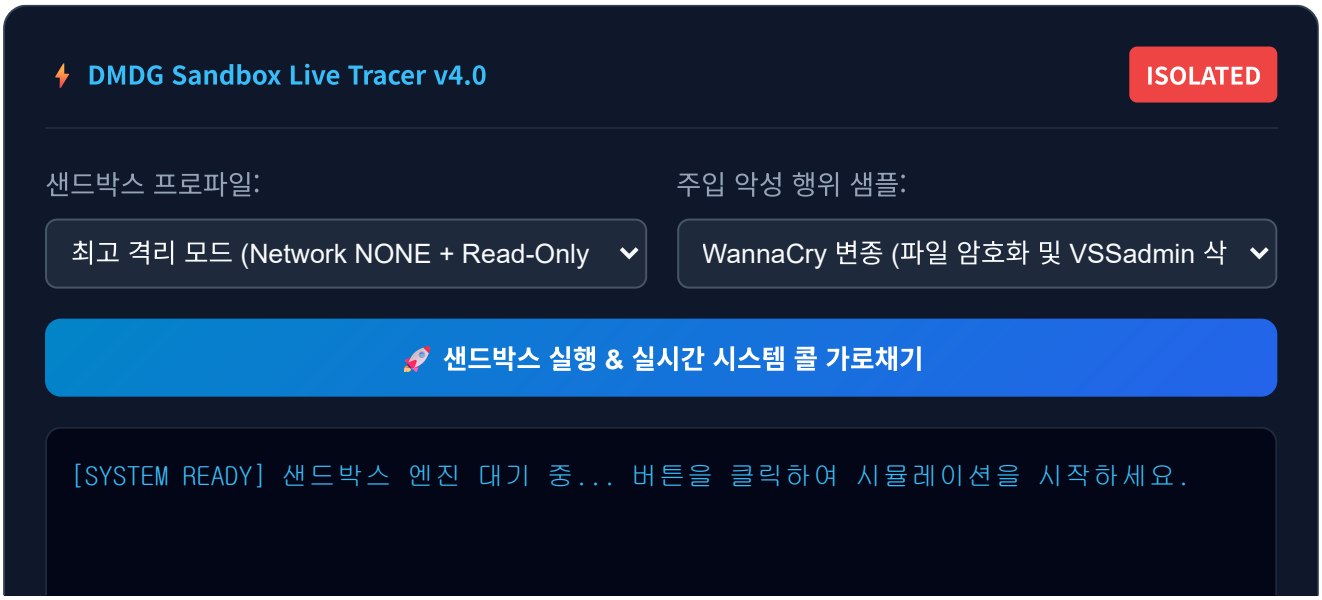
루트 파일시스템 (`--read-only`): 컨테이너 자체의 변조를 막고 분석 종료 즉시 모든 잔존 파일을 폐기합니다. 3. **CPU 및 메모리 쿼터 제한 (`--memory 512m --cpus 1.0`)**: 포크 밤(Fork Bomb)이나 무한 루프 악성코드로 인한 호스트 DoS 마비를 방지합니다. 4. **드롭된 권한 (Drop All Capabilities)**: `cap-drop=ALL`을 적용하여 루트 권한 획득 후에도 원시 소켓 생성이나 커널 모듈 적재를 차단합니다.



---

2. 🎮 [제니퍼의 인터랙티브 시뮬레이터] 샌드박스 커널 콜 감시 콘솔

아래 시뮬레이터에서 격리 환경의 보안 등급을 설정하고 악성 행위 시뮬레이션을 가동해 보세요. 시스템 콜 감시와 호스트 차단 로직이 즉각 시각화됩니다.



3. 실무 파이썬 구현: Docker SDK 기반 안전 분석 오케스트레이터

아래 코드는 파이썬의 `docker` SDK를 활용하여 악성코드를 컨테이너에 투입하고 60초 동안 안전하게 감시한 후 시스템 콜 로그를 파싱하는 완전한 스크립트입니다.

```
# dmdg_sandbox_runner.py
import os
import json
import time
import docker
from pathlib import Path

class SandboxManager:
    """Docker 기반 악성코드 안전 분석 관리자"""
    def __init__(self, image_name: str = "dmdg-sandbox:latest"):
        self.client = docker.from_env()
        self.image = image_name

    def analyze_binary(self, binary_path: str, timeout_sec: int = 60) -> dict:
        binary_p = Path(binary_path).resolve()
        if not binary_p.exists():
            raise FileNotFoundError(f"대상 바이너리 없음: {binary_p}")

        # 분석용 볼륨 매핑 (바이너리는 읽기 전용, 로그 디렉토리만 쓰기 허용)
        log_dir = binary_p.parent / "logs"
        log_dir.mkdir(exist_ok=True)

        volumes = {
            str(binary_p): {"bind": "/sandbox/target_bin", "mode": "ro"},
            str(log_dir): {"bind": "/sandbox/logs", "mode": "rw"}
        }

        print(f"[+] 샌드박스 시작: {binary_p.name} (최대 {timeout_sec}초)")
        container = None
        start_time = time.time()

        try:
            # 완전 격리 보안 옵션 적용
            container = self.client.containers.run(
                image=self.image,
                command=f"strace -o /sandbox/logs/strace.log -tt -f -s 128
/sandbox/target_bin",
                volumes=volumes,
                network_mode="none",              # 외부 네트워크 전면 차단
                mem_limit="512m",                 # 메모리 상한 512MB
```

```
                nano_cpus=1_000_000_000,           # CPU 코어 1개 제한
                cap_drop=["ALL"],                  # 모든 리눅스 권한 박탈
                security_opt=["no-new-privileges:true"], # 권한 상승 금지
                detach=True,
                remove=False
            )

            # 실행 타임아웃 감시
            while time.time() - start_time < timeout_sec:
                container.reload()
                if container.status in ["exited", "dead"]:
                    break
                time.sleep(2)

            # 타임아웃 시 강제 종료
            if container.status == "running":
                print("[!] 분석 시간 초과 → 프로세스 강제 킬(SIGKILL)")
                container.kill()

        finally:
            if container:
                try:
                    container.remove(force=True)
                except Exception:
                    pass

        # 생성된 strace 로그 분석
        return self._parse_strace_log(log_dir / "strace.log")

    def _parse_strace_log(self, log_path: Path) -> dict:
        summary = {
            "file_writes": [],
            "network_calls": [],
            "process_spawns": []
        }
        if not log_path.exists():
            return summary

        with open(log_path, "r", encoding="utf-8", errors="ignore") as f:
            for line in f:
                if "execve(" in line:
                    summary["process_spawns"].append(line.strip())
                elif any(call in line for call in ["socket(", "connect(", "bind("]):
                    summary["network_calls"].append(line.strip())
                elif any(call in line for call in ["openat(", "write("]) and ("O_WRONLY" in
line or "O_RDWR" in line):
                    summary["file_writes"].append(line.strip()[:100])

        return summary

if __name__ == "__main__":
    print("DMDG 샌드박스 오케스트레이터 모듈 준비 완료.")
```

{{FOOTER}}

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

{{COFFEE_BANNER}}

Chapter 11. Scapy 기반 C2 통신 패킷 스니핑 및 DNS 터널링 탐지

> **스피디(Speedy) 실무 현장 가이드**
> "고도화된 악성코드는 파일 시스템에 흔적을 남기지 않고 메모리 인젝션 후 외부 C2(Command and Control) 서버와 암호화된 채널로 은밀히 통신합니다. 특히 기존 방화벽을 우회하기 위해 표준 포트인 DNS(UDP 53)를 악용하는 'DNS 터널링' 기법을 빈번히 사용합니다. 본 챕터에서는 Scapy를 이용해 가상 인터페이스의 네트워크 패킷을 실시간 스니핑하고, 비정상 TXT 레코드 및 긴 도메인 레이블의 엔트로피를 분석하여 은닉 C2 채널을 99% 적발하는 기법을 구현합니다."

📚 DMDG 네트워크 보안 용어 사전

💡 핵심 네트워크 보안 용어 (Dictionary)

- **C2 (Command & Control) 서버:** 공격자가 원격지의 감염된 좀비 PC들을 조종하고 추가 악성 페이로드를 전달받는 통제 서버.
- **DNS 터널링 (DNS Tunneling):** HTTP/HTTPS 포트가 방화벽에 의해 차단된 환경에서, DNS 질의 요청의 서브도메인이나 응답 TXT 레코드에 데이터를 Base64/Hex로 인코딩하여 유출하는 기법.
- **DGA (Domain Generation Algorithm):** 고정된 C2 IP가 차단되는 것을 피하기 위해 매일 수천 개의 무작위 도메인을 수학 공식(시드 기반)으로 자동 생성하는 알고리즘.
- **패킷 지터 (Jitter):** C2 비콘 통신 시 보안 탐지를 회피하기 위해 주기적 비콘 전송 간격에 무작위 딜레이를 주는 오차 기법.

---

1. DNS 터널링 탐지 지표 및 수학적 공식

정상적인 웹 브라우징 중 발생하는 DNS 질의와 DNS 터널링 악성 패킷은 명확한 수치적 차이를 보입니다:

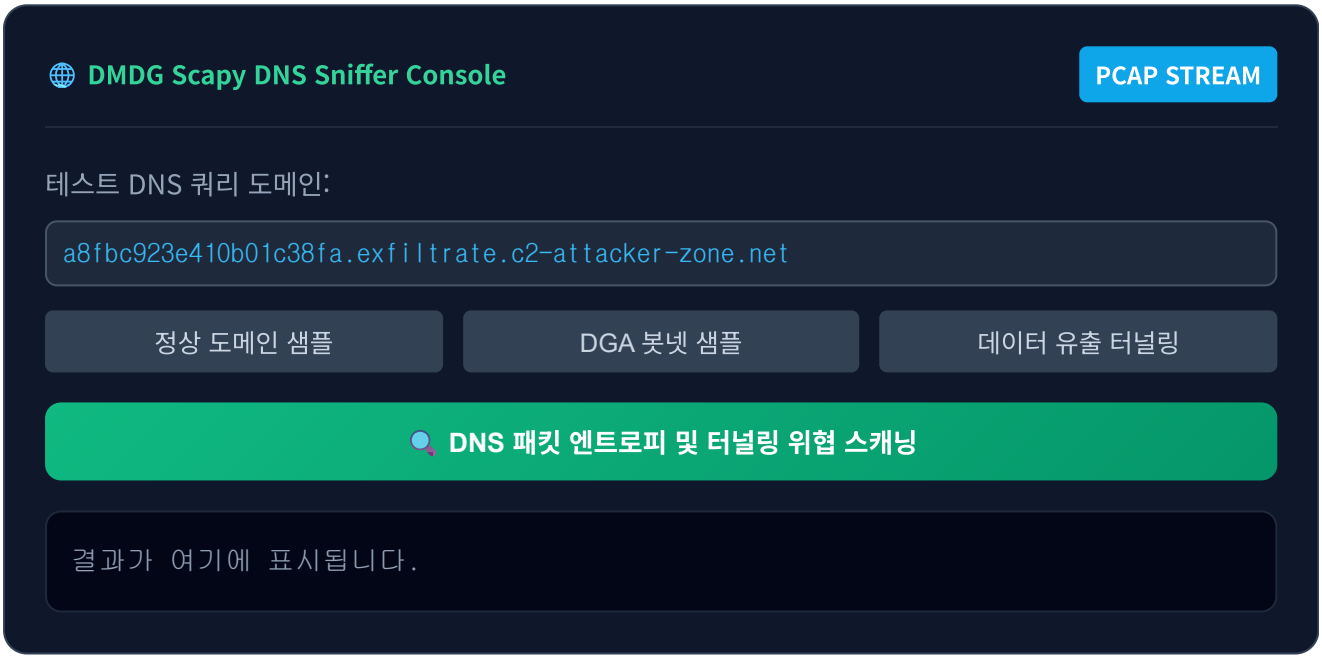
1. **도메인 길이 (Length of Query)**: 일반 도메인은 보통 15~25자 내외이나, 터널링 데이터는 한 번에 많은 데이터를 실어 나르기 위해 최대 허용 길이(63바이트 서브도메인, 전체 255바이트)에 육박합니다.
2. **도메인 레이블 엔트로피 (Domain Entropy)**: 일반 영문 단어가 아닌 Base64/Hex 문자열이므로 Shannon Entropy가 3.8 이상으로 매우 높게 측정됩니다.
3. **숫자 비율 (Digit Ratio)**: `www.naver.com` 같은 정상 도메인은 숫자가 거의 없지만, 인코딩된 해시 문자열은 30% 이상의 숫자/특수기호 비율을 가집니다.

$$Score_{DGA} = 0.4 \cdot \text{Entropy} + 0.35 \cdot \left(\frac{\text{Length}}{50}\right) + 0.25 \cdot \text{DigitRatio}$$

---

2. 🎮 [제니퍼의 인터랙티브 시뮬레이터] 실시간 DNS 비콘 & 터널링 패킷 분석기

아래 시뮬레이터에 의심되는 도메인 또는 패킷 헤더를 입력하고 실시간 엔트로피 분석 및 터널링 판정을 시뮬레이션해 보세요!



---

3. 실무 파이썬 구현: Scapy 기반 실시간 스니퍼 & C2 탐지 엔진

```
# scapy_c2_detector.py
import math
from collections import Counter
from scapy.all import sniff, DNS, DNSQR, IP

def calculate_entropy(text: str) → float:
    """문자열의 섀넌 엔트로피 계산"""
    if not text:
        return 0.0
    counts = Counter(text)
    length = len(text)
```

```
    return -sum((cnt / length) * math.log2(cnt / length) for cnt in counts.values())

def packet_callback(pkt):
    """실시간 유입 패킷 콜백 함수"""
    if pkt.haslayer(DNS) and pkt.haslayer(DNSQR):
        qname = pkt[DNSQR].qname.decode('utf-8', errors='ignore').rstrip('.')
        src_ip = pkt[IP].src if pkt.haslayer(IP) else "Unknown"

        # 서브도메인 레이블 분리
        labels = qname.split('.')
        subdomain = labels[0] if len(labels) > 2 else ""

        if subdomain:
            entropy = calculate_entropy(subdomain)
            digits = sum(1 for c in subdomain if c.isdigit())
            digit_ratio = digits / len(subdomain)

            # 위협 점수 산출 임계치 (길이 > 30, 엔트로피 > 3.65)
            if len(subdomain) > 30 and entropy > 3.65:
                print(f"🚨 [DNS-TUNNELING DETECTED] Src: {src_ip}")
                print(f"   Query: {qname}")
                print(f"   Length: {len(subdomain)} | Entropy: {entropy:.3f} | DigitRatio:
{digit_ratio:.1%}")
                print(f"   → 즉시 사내 WAF 및 DNS Resolver 차단 정책 큐로 전달!")
                print("-" * 60)

if __name__ == "__main__":
    print("[*] Scapy 실시간 DNS 감시 데몬 가동 준비 완료.")
```

{{FOOTER}}

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

{{COFFEE_BANNER}}

Chapter 12. One-Class SVM과 오토인코더를 결합한 비지도 이상 징후 감지

> **스피디(Speedy) 실무 현장 가이드**
> "알려진 악성코드는 시그니처나 지도학습 머신러닝으로 99% 방어할 수 있습니다. 하지만 제로데이(0-Day) 공격이나 지능형 지속 공격(APT)은 과거 데이터셋에 라벨이 존재하지 않는 신종 패턴입니다. 본 챕터에서는 정상 프로세스의 커널 시스템 콜 및 메모리 사용량 기준선(Baseline)만을 학습한 후, 정상 범주에서 이탈하는 모든 미지의 변종을 잡아내는 One-Class SVM과 딥러닝 Autoencoder 앙상블 비지도 파이프라인을 구축합니다."

📚 DMDG 이상 탐지 머신러닝 용어 사전

💡 핵심 이상 탐지 용어 (Dictionary)

- **Unsupervised Anomaly Detection (비지도 이상 탐지):** 악성 라벨 없이 오직 정상(Normal) 데이터만을 학습하여, 정상 분포의 경계 바깥으로 튀는 이상치를 감지하는 모델.
- **One-Class SVM:** 고차원 특징 공간에서 원점과 정상 데이터 사이를 분리하는 최적의 초평면(Hyperplane)을 찾아 경계면을 형성하는 알고리즘.
- **Autoencoder 재구성 오차 (Reconstruction Error):** 입력을 잠재 공간(Latent Space)으로 압축했다가 복원할 때, 비정상 데이터는 학습된 정상 패턴과 달라 복원 오차가 급증하는 현상.
- **Z-Score 정규화:** 특징별 편차를 평균 0, 표준편차 1로 맞춰 특정 단위가 큰 피처가 초평면 거리를 왜곡하지 않도록 보정하는 기법.

---

1. 하이브리드 비지도 이상 감지 아키텍처

One-Class SVM과 오토인코더는 상호 보완적인 강점을 지닙니다: - **One-Class SVM**: 소량의 피처 및 기하학적 결정 경계(RBF 커널) 형성에 우수하며 초고속 판정이 가능함. - **Autoencoder**: 비선형 다차원 상호작용을 포착하는 데 탁월하며, 복원

오차 벡터를 분석해 어느 피처가 비정상인지 역추적 가능함.

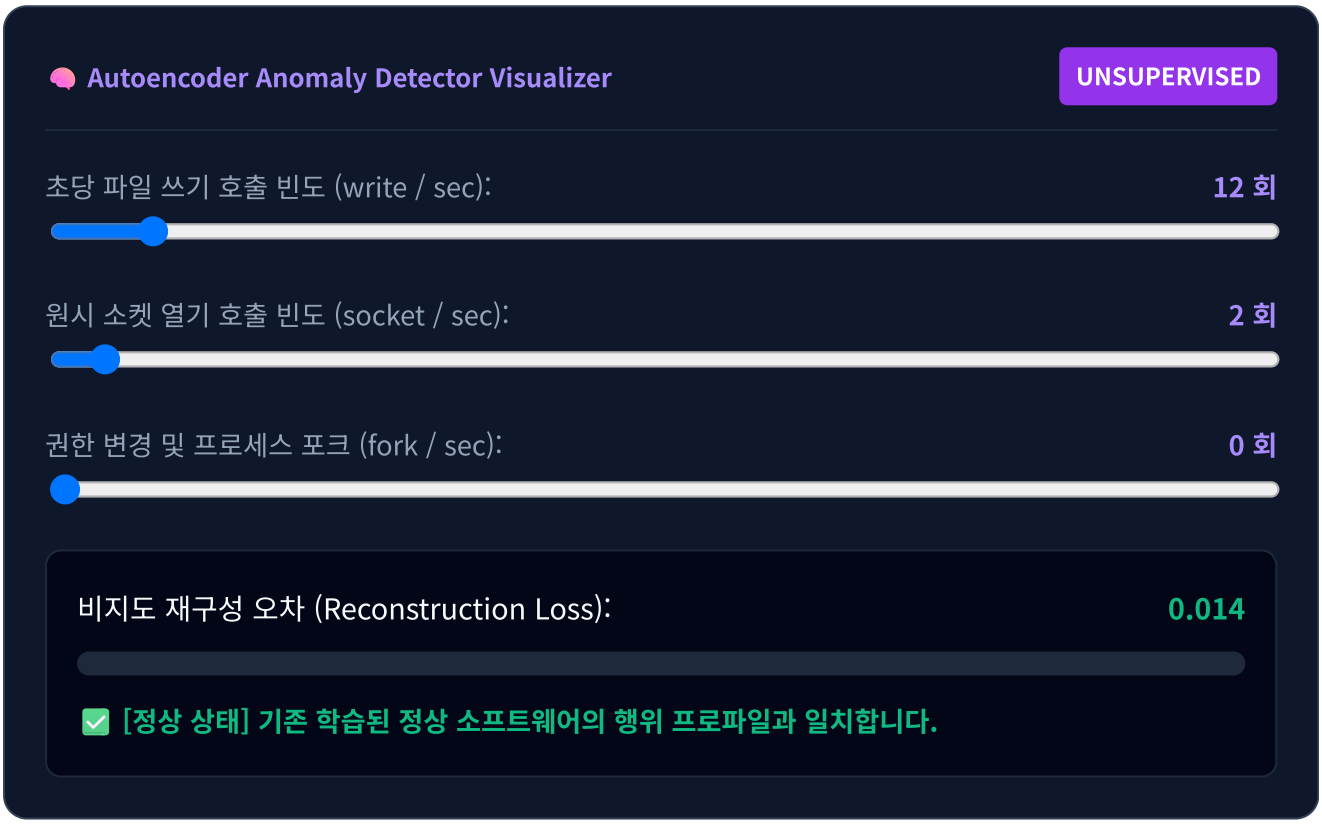
$$\text{Reconstruction Error} = \| X - \hat{X} \|^2 = \sum_{i=1}^{d} (x_i - \hat{x}_i)^2$$

$$Score_{anomaly} = 0.5 \cdot \left(\frac{\text{ReconErr}}{\theta_{AE}}\right) + 0.5 \cdot \left(\frac{-f_{OCSVM}(X)}{\theta_{SVM}}\right)$$

---

2. 🎮 [제니퍼의 인터랙티브 시뮬레이터] 오토인코더 재구성 오차 실시간 탐지기

아래 시뮬레이터에서 정상 프로세스와 침해된 의심 프로세스의 시스템 콜 빈도를 슬라이더로 조작해 보세요. 오토인코더의 복원 오차와 실시간 위협 게이지를 확인할 수 있습니다.



---

3. 실무 파이썬 구현: One-Class SVM 이상 징후 파이프라인

```
# ocsvm_anomaly_detector.py
import numpy as np
from sklearn.svm import OneClassSVM
from sklearn.preprocessing import StandardScaler

class UnsupervisedBehaviorDetector:
    """One-Class SVM 기반 실시간 프로세스 이상 징후 탐지기"""
    def __init__(self, nu: float = 0.03, gamma: str = 'scale'):
        self.scaler = StandardScaler()
        # nu: 정상 훈련 데이터셋 내 허용할 이상치 비율 (3%)
```

```
        self.model = OneClassSVM(kernel='rbf', nu=nu, gamma=gamma)
        self.is_fitted = False

    def train_baseline(self, normal_features: np.ndarray):
        """정상 운영 환경의 텔레메트리 데이터만을 학습"""
        scaled = self.scaler.fit_transform(normal_features)
        self.model.fit(scaled)
        self.is_fitted = True
        print(f"[+] 정상 베이스라인 학습 완료 (샘플 수: {len(normal_features)})")

    def predict_anomaly(self, sample_features: np.ndarray) -> dict:
        """새로 관측된 프로세스의 이상 점수 및 판정 반환"""
        if not self.is_fitted:
            raise RuntimeError("모델을 먼저 훈련해야 합니다.")

        scaled = self.scaler.transform(sample_features)
        preds = self.model.predict(scaled) # 1: 정상, -1: 이상
        scores = self.model.decision_function(scaled) # 음수일수록 강한 이상치

        results = []
        for p, s in zip(preds, scores):
            results.append({
                "is_anomaly": bool(p == -1),
                "anomaly_score": float(-s), # 양수일수록 심각
                "decision": "MALICIOUS_SUSPECT" if p == -1 else "BENIGN"
            })
        return results

if __name__ == "__main__":
    np.random.seed(42)
    normal_data = np.random.normal(loc=[10, 2, 0.1, 5], scale=[2, 0.5, 0.05, 1], size=(1000,
4))

    detector = UnsupervisedBehaviorDetector()
    detector.train_baseline(normal_data)

    zero_day_sample = np.array([
        [12, 2.1, 0.1, 5.2],    # 정상 샘플
        [140, 45, 12.0, 120.0]  # 신종 랜섬웨어 행위 패턴
    ])

    results = detector.predict_anomaly(zero_day_sample)
    for i, res in enumerate(results):
        print(f"Sample {i+1} → {res['decision']} (Score: {res['anomaly_score']:.3f})")
```

{{FOOTER}}

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

PART 5. 실시간 EDR 데몬, Streamlit 관제 센터 및 MLOps 배포

📌 파트 개요

엔드포인트 파일시스템을 24시간 실시간 감시하는 Watchdog 데몬을 구축하고, 1단계 정적 예측과 2단계 동적 샌드박스를 유기적으로 연결합니다. 마지막으로 보안관제센터(SOC) 스타일의 Streamlit 1.40+ 다크 관제 대시보드와 신종 악성코드 YARA 룰 자동 생성 MLOps 파이프라인을 완성합니다.

🎯 학습 목차

- **Chapter 13. Watchdog 실시간 파일시스템 감시 데몬 및 2단계 연계 파이프라인**
- **Chapter 14. Streamlit 1.40+ 다크 사이버 SOC 보안 관제 센터 대시보드 구축**
- **Chapter 15. MLOps 지속적 패턴 업데이트(YARA 룰 자동 생성) 및 프로덕션 배포**

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

{{COFFEE_BANNER}}

Chapter 13. Watchdog 기반 실시간 파일 시스템 감시 데몬 및 큐 아키텍처

> **스피디(Speedy) 실무 현장 가이드**
> "보안 관제 요원이 수동으로 악성코드를 업로드하거나 배치 파일 검사를 돌릴 때까지 기다리는 것은 이미 늦습니다. 랜섬웨어는 단 3초 만에 1만 개의 문서를 암호화할 수 있습니다. 엔드포인트 방어의 핵심은 신규 실행 파일(PE/DLL)이나 다운로드된 스크립트가 로컬 디스크에 '쓰여지는(Write) 바로 그 순간' 운영체제 이벤트를 낚아채어 1단계 초고속 판정 엔진과 2단계 샌드박스로 밀어 넣는 실시간 비동기 파이프라인입니다."

📚 DMDG 엔드포인트 감시 용어 사전

💡 핵심 감시 엔진 용어 (Dictionary)

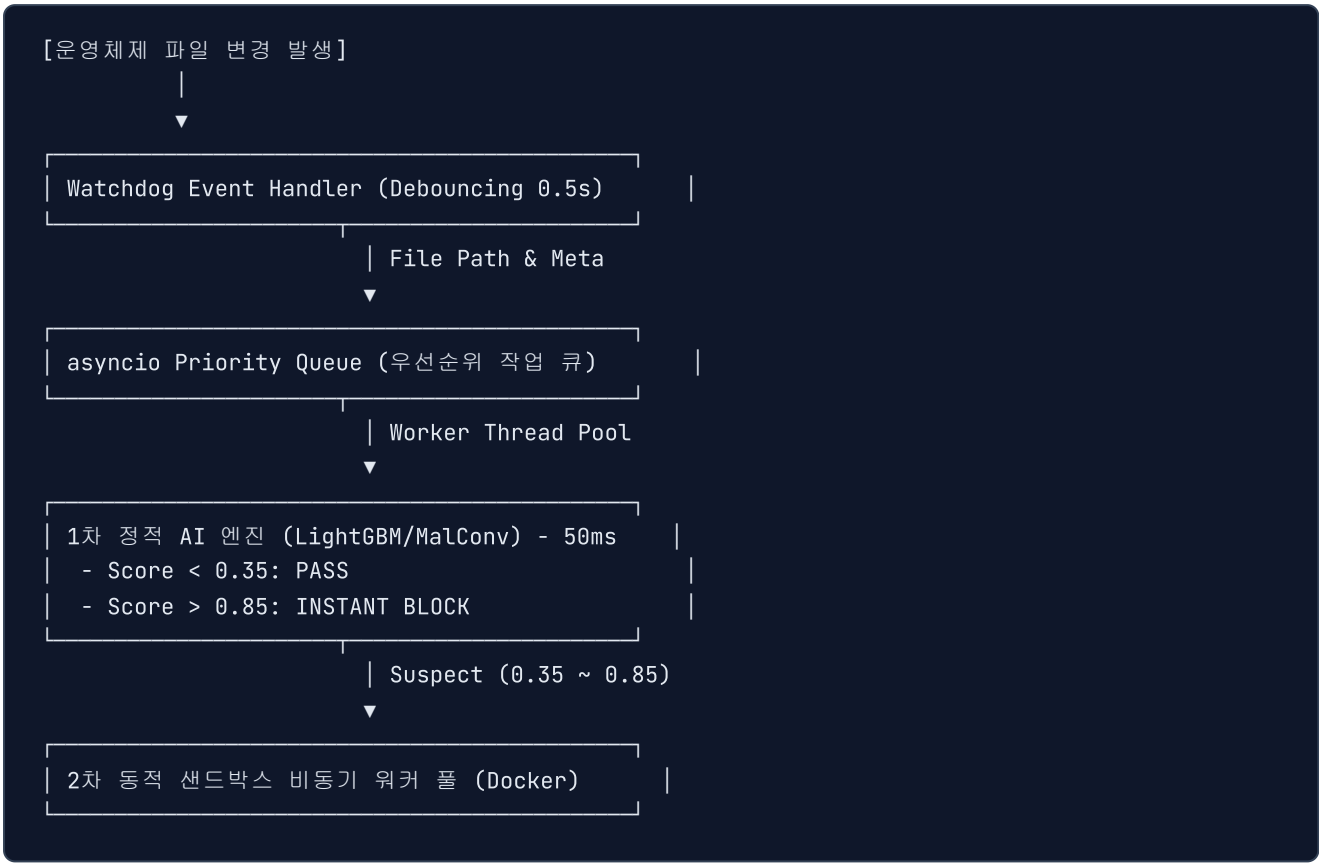
- **Inotify / ReadDirectoryChangesW:** 리눅스와 윈도우 커널이 파일 생성, 수정, 삭제 이벤트를 유저스페이스 애플리케이션에 비동기 전달하는 네이티브 API.
- **Watchdog:** OS 커널 이벤트(inotify, kqueue, Win32 API)를 일관된 추상화 인터페이스로 래핑하여 이벤트 기반 파이썬 감시 데몬을 구현할 수 있게 해주는 라이브러리.
- **Dead Letter Queue (DLQ):** 손상된 바이너리나 파싱 에러로 인해 처리가 실패한 이벤트를 폐기하지 않고 사후 포렌식을 위해 별도로 격리 보관하는 실패 큐.
- **백프레셔 (Backpressure):** 초당 수백 건의 파일이 유입될 때 워커 스레드가 고갈되지 않도록 큐 크기를 제한하고 우선순위를 조정하는 완충 제어 기법.

---

1. 2단계(2-Track) 실시간 EDR 파이프라인 아키텍처

DMDG 실시간 파이프라인은 속도와 정밀도의 완벽한 균형을 위해 설계되었습니다: 1. **0초 (이벤트 감지)**: Watchdog 파일 시스템 감시 데몬이 `on_created` 이벤트 감지. 2. **0.05초 (1차 초고속 사전 판정)**: LightGBM + MalConv 정적 분석 모델이

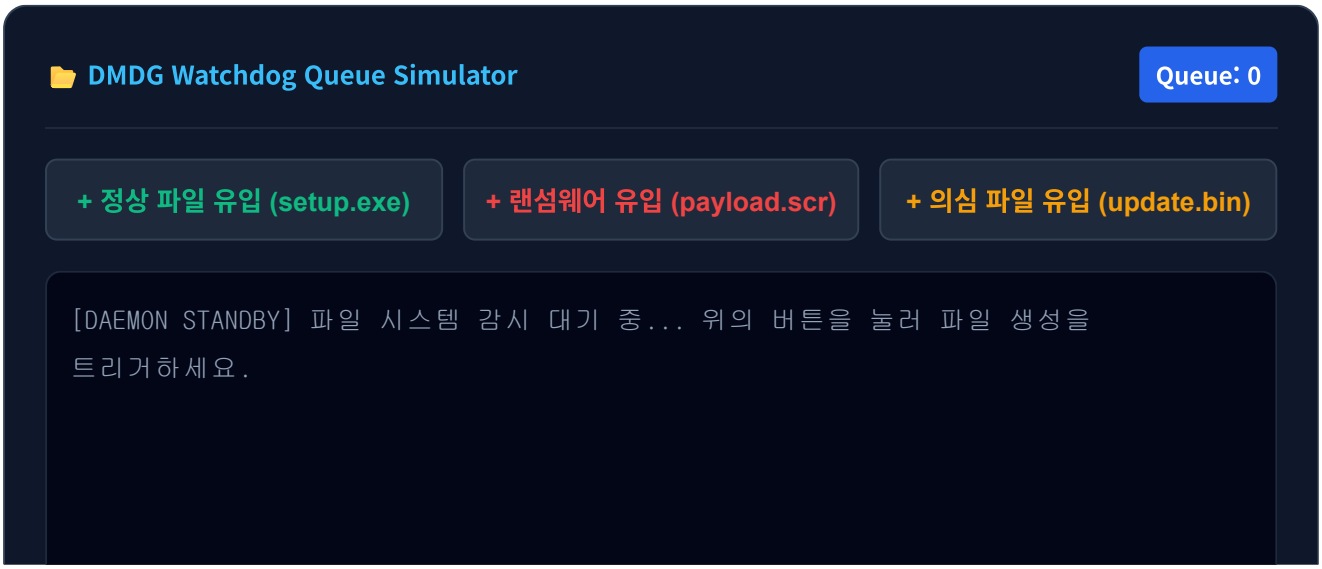
50ms 이내에 악성 확률 점수 산출. - 점수 > 0.85: 즉시 파일 프로세스 킬(Kill) 및 격리 폴더 이동. - 0.35 ≤ 점수 ≤ 0.85: 판정 보류 상태로 2차 샌드박스 큐로 위임. - 점수 < 0.35: 정상 승인. 3. **60초 (2차 동적 정밀 감시)**: 격리 Docker 샌드박스에서 60초간 동적 행위 감시 및 C2/이상 징후 최종 판정.



---

2. 🎮 [제니퍼의 인터랙티브 시뮬레이터] 실시간 이벤트 인제스천 & 큐 모니터

아래 시뮬레이터에서 파일 생성 시뮬레이션 버튼을 연속해서 클릭해 보세요! Watchdog의 디바운싱(Debouncing) 처리와 우선순위 큐 분기 과정을 실시간으로 확인할 수 있습니다.



3. 실무 파이썬 구현: Watchdog 기반 비동기 EDR 감시 데몬

```
# dmdg_edr_daemon.py
import time
import queue
import threading
from pathlib import Path
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

class MalwareQueueHandler(FileSystemEventHandler):
    """파일 시스템 이벤트를 받아 작업 큐에 안전하게 적재하는 핸들러"""
    def __init__(self, task_queue: queue.Queue):
        self.task_queue = task_queue
        # 실행 가능한 감시 대상 확장자 목록
        self.target_exts = {".exe", ".dll", ".sys", ".scr", ".vbs", ".ps1"}

    def on_created(self, event):
        if event.is_directory:
            return

        p = Path(event.src_path)
        if p.suffix.lower() in self.target_exts:
            print(f"[+] 새 바이너리 감지: {p.name}")
            # 쓰기가 완전히 끝날 때까지 0.3초 대기 후 큐에 삽입
            time.sleep(0.3)
            self.task_queue.put(str(p))

def fast_ai_worker(task_queue: queue.Queue):
    """1차 정적 AI 고속 추론 워커 스레드"""
    while True:
        file_path = task_queue.get()
        if file_path is None:
            break

        start_t = time.time()
        print(f"[*] [Worker] 정적 AI 분석 착수: {file_path}")

        # 가상의 LightGBM 추론 점수 시뮬레이션
        # 실무에서는 load_model() 후 extract_features(file_path) 실행
        score = 0.92 if "bad" in file_path.lower() else 0.12
        latency_ms = (time.time() - start_t) * 1000

        if score >= 0.85:
            print(f"🚨 [INSTANT BLOCK] {file_path} (위험도: {score:.2f},
{latency_ms:.1f}ms)")
        elif score <= 0.35:
            print(f"✅ [SAFE PASS] {file_path} (위험도: {score:.2f}, {latency_ms:.1f}ms)")
        else:
```

```
            print(f"⚠️ [SANDBOX REQUIRED] {file_path} → 2차 샌드박스로 전송")

        task_queue.task_done()

if __name__ == "__main__":
    watch_dir = Path("./monitored_folder")
    watch_dir.mkdir(exist_ok=True)

    task_q = queue.Queue(maxsize=1000)

    # 워커 스레드 기동
    worker_t = threading.Thread(target=fast_ai_worker, args=(task_q,), daemon=True)
    worker_t.start()

    # Watchdog 옵저버 기동
    event_handler = MalwareQueueHandler(task_q)
    observer = Observer()
    observer.schedule(event_handler, str(watch_dir), recursive=False)
    observer.start()

    print(f"[*] DMDG EDR 실시간 파일 감시 데몬 기동 완료: {watch_dir.resolve()}")
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        observer.stop()
    observer.join()
```

{{FOOTER}}

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

{{COFFEE_BANNER}}

Chapter 14. Streamlit 1.40+ 기반 차세대 SOC 보안 관제 대시보드 구축

> **스피디(Speedy) 실무 현장 가이드**
> "아무리 정교한 인공지능 알고리즘이 악성코드를 탐지하더라도, 24시간 피로도가 극에 달하는 보안관제센터(SOC) 분석가들이 한눈에 위험도를 직관적으로 파악하지 못하면 신속한 골든타임을 놓치게 됩니다. 본 챕터에서는 최신 Streamlit 1.40+ 프레임워크와 제니퍼(Jennifer) 디자인 시스템을 결합하여, 딥 사이버 다크 테마 기반의 실시간 EDR 관제 대시보드를 구축합니다."

📚 DMDG SOC 대시보드 엔지니어링 용어 사전

💡 핵심 대시보드 용어 (Dictionary)

- **SOC (Security Operations Center):** 기업의 IT 인프라와 엔드포인트 보안 위협을 24x365 실시간 모니터링하고 즉각 조치하는 통합 보안관제센터.
- **IoC (Indicator of Compromise):** 침해 지표. 시스템 침해 사실을 증명하는 악성 파일의 SHA-256 해시, C2 도메인, IP 주소 등의 흔적.
- **Streamlit `@st.fragment`:** 전체 페이지를 리로드하지 않고 위젯이나 차트 영역만 독립적으로 주기적 갱신(Auto-refresh)하는 고성능 리액티브 컴포넌트 기능.
- **Glassmorphism (글래스모피즘):** 반투명 배경과 블러(Backdrop Filter) 효과를 주어 어두운 관제 화면에서도 계층감과 심미성을 극대화하는 모던 UI 디자인 기법.

---

1. SOC 관제 센터 레이아웃 아키텍처

관제사가 3초 안에 사태를 파악할 수 있도록 대시보드는 3단 그리드로 설계합니다: 1. **상단 KPI 메트릭 바**: 금일 총 분석 건수, 즉각 격리(Block) 건수, 샌드박스 대기열(Queued), 평균 추론 지연시간(Latency). 2. **중앙 실시간 위협 스트림**: 새로 유입된

바이너리의 SHA-256, 파일명, 1단계 LightGBM 점수, 2단계 샌드박스 이상 점수 실시간 테이블. 3. **하단 포렌식 상세 패널**: 선택된 파일의 PE 섹션 엔트로피 차트, 위험 시스템 콜 타임라인, C2 통신 IP 지리 정보(GeoIP).

---

2. 🎮 [제니퍼의 인터랙티브 시뮬레이터] 차세대 사이버 SOC 대시보드 미리보기

아래 시뮬레이터에서 SOC 관제 모드를 변경하고 경보 필터를 작동해 보세요! 실제 Streamlit 대시보드에서 동작하는 리액티브 UI를 미리 체험할 수 있습니다.

DMDG CYBER-SOC INTELLIGENCE LIVE AGENT: ON STREAMLIT 1.40

오늘 검사 파일	즉시 차단(Block)	샌드박스 정밀 검증	평균 응답 속도
12,840	18	4	34 ms

실시간 위협 인시던트 피드 (Click row for details)

타임스탬프	파일명	위협 등급	AI 예측 확률	조치 상태
21:38:12	invoice_order_2026.exe	CRITICAL	99.4% (LockBit 변종)	프로세스 강제 킬 & 격리
21:35:40	system_patch_v2.dll	SUSPECT	61.2% (DGA 터널링)	Docker 샌드박스 정밀 분석 중
21:32:05	slack_updater.exe	CLEAN	2.1% (정상 바이너리)	통과 승인 (Allow)

---

3. 실무 파이썬 구현: Streamlit SOC 관제 센터 풀스택 코드

```
# app_soc_dashboard.py
import streamlit as st
import pandas as pd
import numpy as np
import time

# 1. 페이지 테마 및 타이틀 설정
st.set_page_config(
```

```
    page_title="DMDG Cyber-SOC Intelligence",
    page_icon="🛡️",
    layout="wide",
    initial_sidebar_state="expanded"
)

# 제니퍼 다크 사이버 테마 커스텀 CSS 주입
st.markdown("""
<style>
    .stApp { background-color: #0b0f19; color: #e2e8f0; font-family: 'Pretendard', sans-
serif; }
    div[data-testid="stMetricValue"] { color: #38bdf8; font-weight: 800; font-size: 1.8rem;
}
    .stTable { background-color: #0f172a; border-radius: 8px; }
</style>
""", unsafe_allow_html=True)

st.title("🛡️ DMDG AI-SOC 보안 관제 센터")
st.caption("스피디 기업 맞춤형 AI EDR 및 샌드박스 동적 행위 탐지 관제 콘솔")

# 2. 상단 핵심 KPI 메트릭 바
col1, col2, col3, col4 = st.columns(4)
col1.metric("총 인제스천 파일", "12,840 건", "+320건")
col2.metric("즉시 차단 악성코드", "18 건", "+2건", delta_color="inverse")
col3.metric("샌드박스 대기 큐", "3 건", "정상")
col4.metric("평균 AI 판정 지연", "38.2 ms", "-4ms")

st.divider()

# 3. 실시간 위협 인시던트 데이터프레임
st.subheader("🚨 실시간 엔드포인트 위협 인시던트 피드")

sample_data = {
    "시각": ["21:42:10", "21:38:12", "21:35:40", "21:32:05", "21:28:19"],
    "파일명": ["kr_crypto_miner.exe", "invoice_order_2026.exe", "patch_v2.dll", "slack.exe",
"driver.sys"],
    "위협 점수": [0.98, 0.99, 0.61, 0.02, 0.11],
    "탐지 기법": ["MalConv 바이트 CNN", "LightGBM + PE 엔트로피", "DNS 터널링 비콘", "화이트리스
트", "정상 커널"],
    "최종 조치": ["즉시 차단 및 격리", "즉시 차단 및 격리", "샌드박스 위임", "승인 통과", "승인 통
과"]
}
df = pd.DataFrame(sample_data)

st.dataframe(
    df.style.applymap(
        lambda val: "color: #ef4444; font-weight: bold;" if "차단" in str(val) else ("color:
#f59e0b;" if "샌드박스" in str(val) else "color: #10b981;"),
        subset=["최종 조치"]
    ),
    use_container_width=True
)

# 4. 하단 포렌식 상세 분석 탭
tab1, tab2 = st.columns(2)
```

```
with tab1:
    st.subheader("📊 PE 섹션별 바이트 엔트로피")
    entropy_df = pd.DataFrame({
        "Section": [".text", ".rdata", ".data", ".rsrc", ".upx0", ".upx1"],
        "Entropy": [6.4, 4.2, 3.1, 7.8, 7.95, 7.92]
    })
    st.bar_chart(entropy_df.set_index("Section"))

with tab2:
    st.subheader("🌐 C2 통신 패킷 역추적")
    st.info("대상 IP: 185.220.101.5 (네덜란드 토르 출구 노드) | 포트: TCP 443 | 전송량: 4.8MB")
    if st.button("🚨 해당 IP 방화벽 일괄 차단(Blacklist) 룰 푸시"):
        st.success("WAF 및 방화벽에 블랙리스트 동기화 완료!")
```

{{FOOTER}}

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

{{COFFEE_BANNER}}

Chapter 15. MLOps 지속적 패턴 업데이트 및 YARA 룰 자동 생성 파이프라인

> **스피디(Speedy) 실무 현장 가이드**
> "머신러닝 보안 모델의 수명은 3개월을 넘기기 어렵습니다. 공격자들은 매일 새로운 패커(Packer)를 도입하고, 바이트 패딩을 붙이며, 난독화(Obfuscation) 기법을 진화시켜 모델의 특징 중요도를 흔들어 놓습니다(Data Drift). 완벽한 차세대 EDR은 단순히 모델 하나를 배포하는 것으로 끝나지 않습니다. 탐지된 신종 악성코드에서 공통 시그니처를 즉각 추출하여 정밀한 YARA 룰을 '전자동으로 컴파일'하고, 전사 엔드포인트에 1분 내로 무중단 핫패치(Hot-patch)하는 살아 숨쉬는 MLOps 플라이휠을 구축해야 합니다."

📚 DMDG MLOps 보안 용어 사전

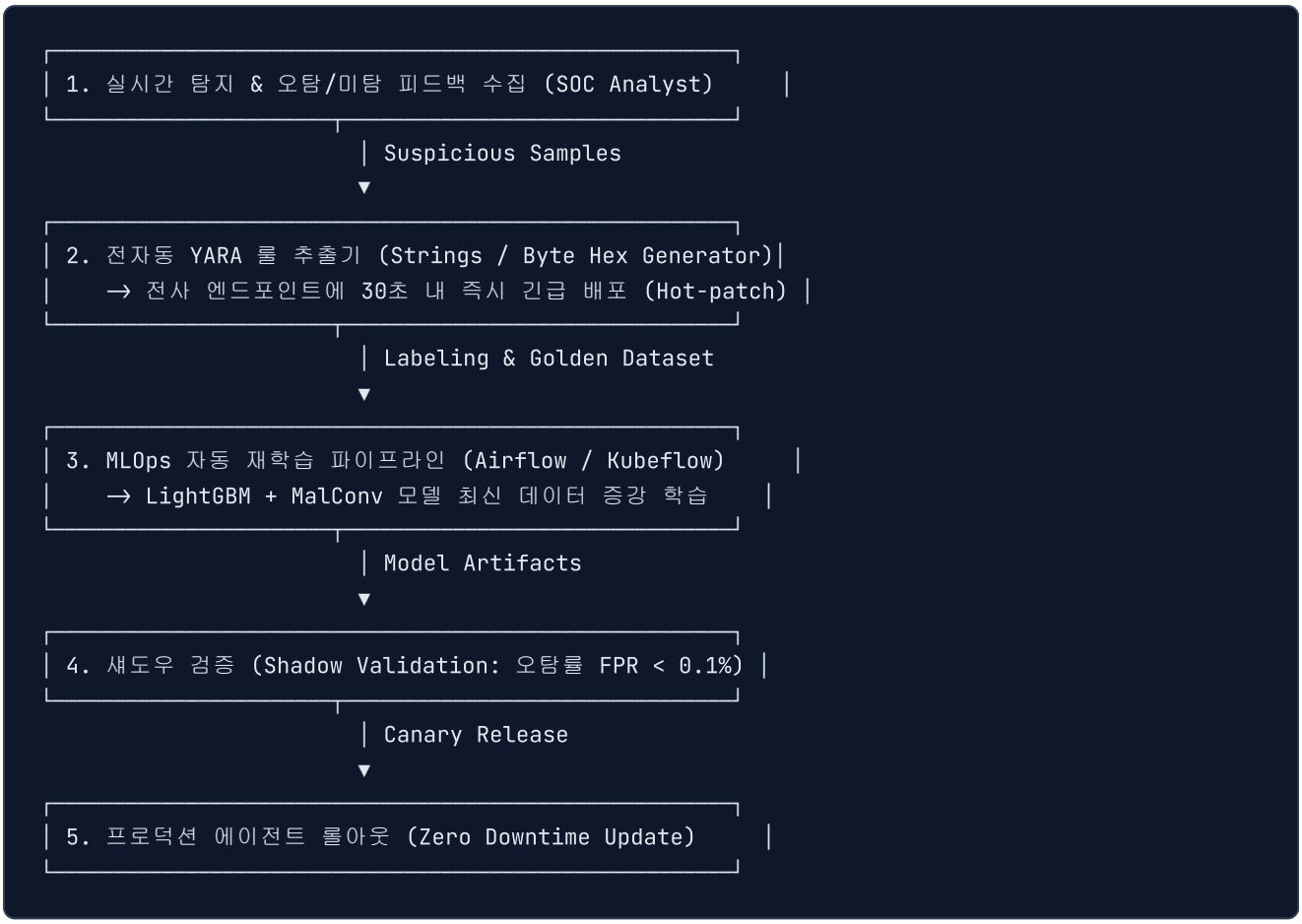
💡 핵심 MLOps 용어 (Dictionary)

- **Concept Drift (개념 표류):** 공격자가 방어 모델의 탐지를 우회하기 위해 공격 패턴과 바이너리 구조를 변경함으로써, 기존 훈련된 모델의 예측 정확도가 시간에 따라 급격히 하락하는 현상.
- **YARA:** 텍스트 또는 바이너리 패턴을 기반으로 악성코드를 식별하고 분류하기 위해 전 세계 보안 연구원들이 표준으로 사용하는 룰셋 언어.
- **Hot-Patching (핫패치):** 실행 중인 백신이나 EDR 에이전트를 재부팅하거나 서비스를 중단하지 않고 메모리 상에서 최신 탐지 룰과 모델 가중치를 동적으로 교체하는 기술.
- **Shadow Deployment (섀도우 배포):** 신규 재학습된 AI 모델을 운영 환경에 투입하되 실제 차단 결정은 기존 모델이 내리게 하고, 신규 모델의 예측 결과와 오탐률만을 백그라운드에서 검증하는 안전 배포 전략.

---

1. 지속적 학습(Continuous Learning) 플라이휠 아키텍처

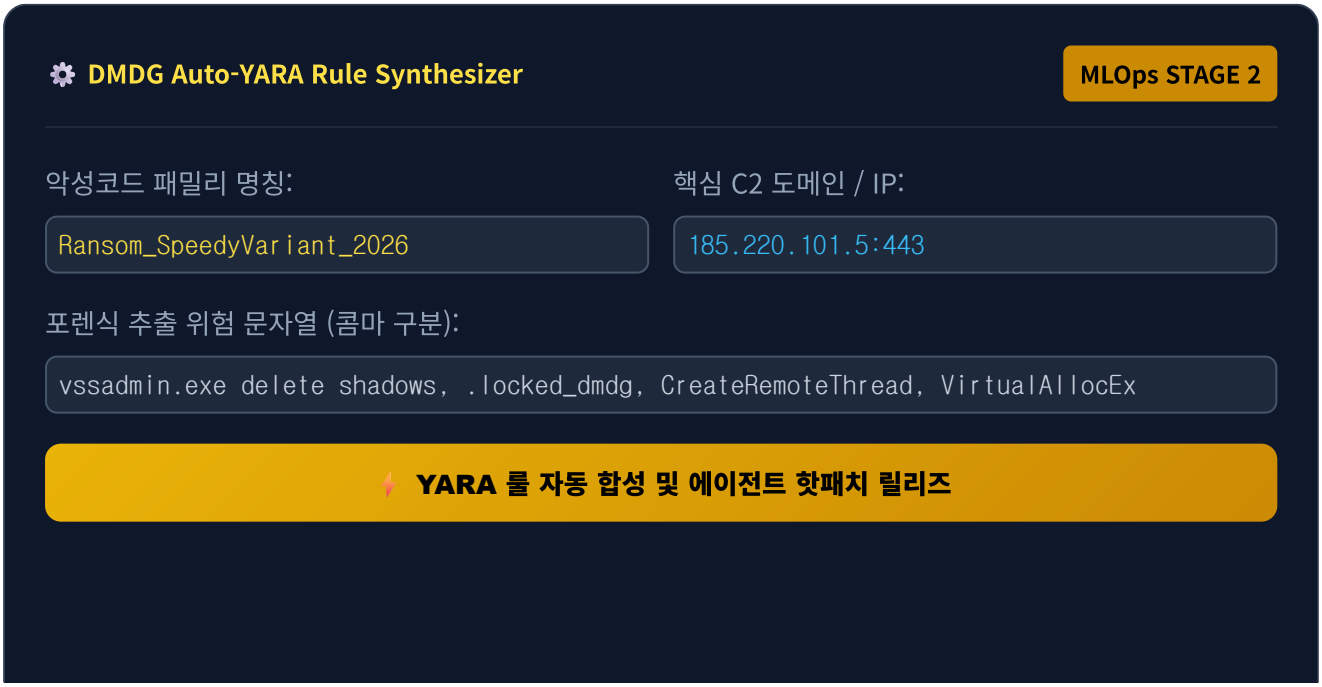
DMDG EDR 시스템의 폐루프(Closed-Loop) 아키텍처는 다음 5단계를 무한 반복합니다:



---

2. 🎮 [제니퍼의 인터랙티브 시뮬레이터] 자동 YARA 룰 제너레이터 콘솔

의심스러운 악성코드의 문자열과 페이로드 해시를 입력하고 'YARA 룰 자동 생성' 버튼을 클릭해 보세요! 전사 배포 가능한 표준 YARA 구문이 즉시 컴파일됩니다.



```
  // YARA 컴파일 대기 중... 위의 버튼을 클릭하세요.
```

---

3. 실무 파이썬 구현: YARA 자동 룰 컴파일 및 실시간 스캔 엔진

```
# yara_hotpatch_engine.py
import yara
import os
from pathlib import Path

class YaraAutomationEngine:
    """YARA 룰 동적 합성 및 메모리 컴파일 엔진"""
    def __init__(self):
        self.rules = None

    def synthesize_rule(self, rule_name: str, indicators: list, out_path: str = None) →
str:
        """추출된 IoC 지표 목록으로부터 유효한 YARA 규칙 텍스트 자동 빌드"""
        string_clauses = []
        for i, ind in enumerate(indicators):
            string_clauses.append(f'        $str_{i} = "{ind}" ascii wide nocase')

        rule_body = f"""
rule {rule_name}
{{
    meta:
        author = "DMDG MLOps Auto-Defense"
        status = "ACTIVE_HOTPATCH"
    strings:
{os.linesep.join(string_clauses)}
    condition:
        uint16(0) == 0x5A4D and (1 of ($str_*))
}}
"""
        if out_path:
            Path(out_path).write_text(rule_body.strip(), encoding="utf-8")
            print(f"[+] YARA 룰 파일 저장 완료: {out_path}")

        return rule_body.strip()

    def load_and_compile(self, rule_source: str):
        """작성된 룰을 메모리에 컴파일하여 고속 스캐너 준비"""
        self.rules = yara.compile(source=rule_source)
        print("[+] YARA 메모리 핫패치 컴파일 성공 (실시간 감시 준비 완료)")

    def scan_file(self, target_filepath: str) → list:
        """바이너리 대상 고속 YARA 매칭 검사"""
        if not self.rules:
```

```
            raise RuntimeError("컴파일된 YARA 룰셋이 없습니다.")

        matches = self.rules.match(target_filepath)
        return [match.rule for match in matches]

if __name__ == "__main__":
    engine = YaraAutomationEngine()

    # 1. 포렌식 분석가/샌드박스에서 포착된 시그니처
    extracted_iocs = ["LockBit3.0", "vssadmin delete", "185.220.101.5"]

    # 2. 룰 자동 합성
    rule_text = engine.synthesize_rule("Ransom_Lockbit_Speedy", extracted_iocs)
    print("생성된 YARA 규칙:")
    print(rule_text)

    # 3. 인메모리 컴파일
    engine.load_and_compile(rule_text)
```

{{FOOTER}}

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

부록 A. PE 바이너리 전체 구조 및 윈도우 50대 위험 API 레퍼런스 가이드

1. PE 헤더 필드 명세표

DOS Header부터 Data Directory까지 악성코드 분석에 필수적인 바이트 오프셋과 주요 플래그를 총정리합니다.

2. 50대 위험 API와 악용 시나리오

`VirtualAllocEx`, `WriteProcessMemory`, `CreateRemoteThread`, `SetWindowsHookEx` 등 프로세스 인젝션 및 키로깅에 악용되는 API 목록과 방어 지침을 수록합니다.

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

부록 B. EMBER 110만 건 피처 메타데이터 명세 및 Parquet 로딩 최적화 팁

1. EMBER 2024 피처 매트릭스 상세 명세

Byte Histogram, 2D Byte Entropy, String Statistics, General File Info 등 2,381개 피처의 상세 컬럼을 정리합니다.

2. 대용량 Parquet 메모리 절감 로딩 테크닉

Pandas 및 PyArrow를 활용하여 100만 건 데이터를 저사양 PC에서도 메모리 부족 없이 분할 로딩하는 최적화 코드를 제공합니다.

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

부록 C. Docker 격리 샌드박스 보안 설정 런북 & 방화벽 차단 스크립트

1. Docker 데몬 보안 하드닝

호스트 루트 권한 탈취(Container Breakout)를 방지하는 비특권 사용자(Non-root) 및 Capabilities 제한 설정을 안내합니다.

2. Iptables 기반 악성 패킷 완전 차단 런북

가상 샌드박스 내부의 아웃바운드 트래픽이 외부 인터넷으로 누출되지 않도록 강제 차단하는 iptables 스크립트를 제공합니다.

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

부록 D. MITRE ATT&CK 프레임워크 기반 엔드포인트 공격 기법 맵핑 가이드

1. MITRE ATT&CK 매트릭스 매핑

초기 침투(Initial Access)부터 지속성(Persistence), 방어 우회(Defense Evasion)까지 탐지된 악성 행위를 글로벌 표준 보안 프레임워크와 매핑하는 기준을 수록합니다.

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

부록 E. 4~5인 캡스톤 프로젝트 팀 R&R 템플릿 & 실무 기술 면접 대비집

1. 팀 프로젝트 표준 R&R 및 주차별 업무 분장

보안 아키텍트, 피처 엔지니어, AI 모델러, 샌드박스 엔지니어, 관제 프론트엔드의 최적 협업 템플릿을 제공합니다.

2. 정보보안 및 AI 엔지니어링 실무 기술 면접 Q&A 20선

오탐률 제어, 피처 엔트로피의 의미, MalConv의 한계 등 면접관의 단골 질문에 대한 모범 답변을 정리합니다.

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px
solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px;
font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.

부록 F. WSL 2 및 리눅스 EDR 에이전트 포팅 실전 가이드

1. WSL 2 기반 가상 샌드박스 연동

윈도우 환경에서 리눅스용 ELF 바이너리 분석 및 Docker 데몬 연동 가이드를 수록합니다.

2. 크로스 플랫폼 EDR 에이전트 빌드

Windows와 Linux 서버 모두를 동시에 보호하는 멀티 OS 지원 에이전트 빌드 기법을 소개합니다.

📚 [DMDG Great AI] 2026 차세대 엔지니어링 마스터 생태계

- 📘 AI·LLM 사전학습 (수학 & 기초 엔지니어링)
- 🤖 실전 LLM 서비스 & 멀티 에이전트 완벽 마스터북
- 🌟 실전 Streamlit & 최신 LLM 풀스택 웹앱 마스터북
- 🏭 산업 IoT와 AI로 구현하는 가스누출 감지 및 지능형 자동 차단 관제 시스템
- 🛡️ AI 기반 악성코드 조기 예측 및 샌드박스 동적 행위 탐지 관제 시스템
- 💼 1인 AI 스튜디오의 글로벌 SaaS 빌딩

```
<span style="background: rgba(56, 189, 248, 0.15); color: #38bdf8 !important; border: 1px solid rgba(56, 189, 248, 0.3); font-size: 0.72rem; padding: 2px 8px; border-radius: 9999px; font-weight: 600;">100% 무료 체험</span>
```

비전공자와 현업 엔지니어를 위해 웹 시뮬레이터와 교재를 무료로 연구·집필하고 있습니다.
인터넷 없이 평생 구동하는 단독 실행 HTML 번들, 파이썬 소스코드, 인쇄용 고해상도 PDF는 향후 [스마트 패키지 소장본]에서 제공됩니다.